

基于信息约束与非对称结算的光伏微网储能滚动优化研究

摘要

本文针对光伏发电功率、居民负荷与购电价格随时间变化条件下的微网经济调度问题，研究储能如何在满足供电平衡、容量与功率边界及充放电效率的前提下跨时段转移电量。四问的差异依次来自信息是否已知、计划能否更新以及偏差如何结算。因此，本文从实际能量流向出发，先建立每个 10 分钟时段的电量平衡和储电量递推关系，再按信息到达过程逐层构造可执行的调度模型。

针对问题一：全天电价、负荷与光伏曲线均已知，决策只需在 144 个时段间安排购电和储能。由母线守恒、储电量递推及容量功率边界建立确定性混合整数线性规划，并以二元状态排除同时充放电。**最优日购电量为 59482.70kWh，购电费用为 35126.95 元，较无储能基线降低 26.90%；**连续松弛得到相同目标值，且能量平衡与状态转移残差均低于 10^{-12} kWh。

针对问题二：每日 0:00 只能依据历史数据制定全天计划，实际负荷与光伏在计划冻结后才揭示，由此形成严格的“先计划、后实现”两阶段结构。第一阶段确定正常购电与储能动作，第二阶段不再调整储能，仅按 5 倍电价紧急购电补足缺口；利用决策日前的整日净负荷残差构造场景，并以 48 小时延伸窗传递日末储能价值。**2 月 1 日至 12 月 31 日总费用为 15258022.89 元，紧急购电量为 404258.73kWh。**

针对问题三：0:00、6:00、12:00 和 18:00 的新光伏预报只改变未执行时段的信息，已发生的购电、充放电和费用不能回写。因此在每个发布时刻固定当前储电量，重算剩余时域并执行至下一次更新，形成滚动混合整数线性规划；增购、减购与最终紧急购电均按题定规则逐次记账。**四次更新的全年总费用为 15731687.04 元，较仅 0:00 计划减少 659704.73 元，紧急购电量下降 28.03%。**

针对问题四：附件中的实时价格是事后实现值，决策时不能预先读取。本文仅用历史价格形成因果预测，分别驱动问题二和问题三，得到全年总费用 **16124373.46 元和 16550967.30 元**。为区分可执行结果与事后评价，另计算完全价格信息反事实基准及同时知道全年价格、负荷和光伏的严格下界 12780453.36 元。全部模型均通过跨日储电量、充放电互斥、费用账本和未来信息泄漏检查。

关键词： 光伏微网；储能调度；两阶段随机规划；模型预测控制；因果预测；混合整数线性规划

一、问题重述

1.1 问题背景

随着分布式光伏发电技术的推广，光伏电源逐渐应用于居民小区、园区和楼宇等负荷中心。微网能够协调分布式电源、储能设备、用户负荷与外部电网之间的能量交换，从而提高光伏电能的本地消纳能力，降低对外部电网的依赖。由于光伏发电功率具有波动性，居民负荷随时间变化，外部电网电价也可能存在峰谷差异，因此如何合理安排购电与储能充放电，成为影响微网经济性和供电可靠性的关键问题。

储能设备具有跨时段调节能力：在光伏发电功率富余或电价较低时充电，并在光伏不足或电价较高时放电，从而实现能量的跨时段调节。但储能状态受到容量上下限、最大充放电功率和充放电效率等物理条件的约束，使得当前的调度决策会直接影响后续时段的可行性与运行成本。因此，微网购电策略不能只依据当前时刻的电价和供需关系进行判断，还需综合考虑全天负荷、光伏发电功率、储能状态及未来信息的不确定性，在满足负荷需求的同时降低购电费用。

本题以某微网系统为研究对象，给出了外部电网电价、小区负荷、光伏实际功率及光伏发电功率预测功率等数据，并规定储能设备的额定容量为 12000kWh，储电量须保持在 1200~10800kWh 之间，最大充放电功率均为 5000kW，充放电效率为 90%。题目以 10min 为一个调度时段，并设置了确定性预测、日内信息更新、滚动调整和波动电价等情形，要求分别制定相应的购电及储能调度策略。

此外，在实际运行中，光伏发电功率、负荷及外部电价均随时间变化，计划与实际之间的偏差可能引发紧急购电或调整购电费用，因此需要根据实时信息动态优化储能调度与购电策略。综上所述，本题的核心是建立能够描述微网能量平衡、储能状态转移、购电结算和信息更新过程的数学模型，在不同预测信息和价格机制下，制定满足供电约束、储能约束且具有较低运行成本的调控方案。

1.2 基本问题

基于上述背景，本文利用题目所给的电价、居民负荷、光伏发电及其预测数据，结合储能设备的运行参数和购电结算规则，逐步建立微网购电与储能调度模型，依次建立数学模型解决以下四个相互递进的问题：

问题一：在某日 144 个时段的电价、居民负荷和光伏发电功率预测数据均已知的条件下，确定外网购电及储能充放电计划，使当日购电费用最小，并对结果进行解释和可行性检验。

问题二：在每天 0:00 仅利用历史数据制定当日计划，而实际小区负荷和光伏发电功率随后逐步揭示的条件下，确定计划购电及储能充放电方案；若微网提供的电能低于小区负荷，则按交易时刻电价的 5 倍进行紧急购电，并计算 2025 年 2 月 1 日至 12 月 31 日的总费用。

问题三：在问题二的基础上，利用每天 0:00、6:00、12:00 和 18:00 发布的未来 24h 光伏发电功率预报，对尚未执行时段的购电及储能计划进行调整；按照增购、减购和紧急购电规则结算，并分析是否需要引入其他时刻的预报制定调整购电策略。

问题四：当外网电价变为事前未知的实时波动价格时，分别按照问题二和问题三的信息更新方式制定购电策略，计算相应的费用，并比较不同信息更新机制下的调度效果。

1.3 关键约束与实际意义

微网调度并非简单地确定光伏发电功率、储能与外部电网的供电顺序，而是需要受三重约束的共同作用：一是**物理约束**，各时段的供电量应满足小区负荷需求，储能设备还受到储电量上下限、充放电功率上限及充放电效率的约束；二是**信息约束**，不同问题中负荷、光伏发电功率和电价的可知范围不同，决策时刻能够使用的信息集也相应变化；三是**结算约束**，计划购电量与实际需求之间的差额，需按照题目规定的价格规则进行结算。三重约束中任一变化都会影响调度模型的目标函数或约束形式，改变模型结构，因此本文将四个问题视为同一微网调度模型在不同信息条件和结算规则下的逐步扩展，而非彼此独立的四个问题。

二、问题分析

本文研究由光伏、储能、外网和居民负荷构成的微网调度问题。采用相同的母线能量平衡关系、储能状态转移以及容量、功率和充放电互斥边界；真正逐时变化的是决策时刻能够获得的信息、计划是否允许更新以及偏差如何结算。因此，本文不把四问处理为彼此独立的算法，而是在统一物理骨架上依次加入不确定性、滚动信息和实时价格，并按“信息集—决策—状态实现—费用结算—结果检验”的闭环展开。

2.1 问题一

问题一给出同一典型日内 144 个时段的电价、居民负荷和光伏发电功率，要求安排外网购电及储能充放电，使全天购电费用最低。功率以 kW 给出，而购电量、储电量和费用均按 kWh 计算，故首先将每个 10min 时段的功率乘以 1/6h，统一时间标签、功率和电量的对应关系。

从能量流看，外网购电、光伏和储能放电构成母线输入，居民负荷、储能充电和弃电构成母线输出；相邻时段由储电量递推连接。题目中的费用、功率边界和状态转移均为线性关系，只有充放电互斥需要二元变量，因此建立确定性混合整数线性规划。问题一的日内曲线在决策前完全已知，且题目明确要求首尾储电量相等，故可一次求得 144 个时段的全日方案。求解后以无储能基线衡量经济收益，以连续松弛给出目标下界，并逐项复算能量平衡、SOC 边界和费用台账。

2.2 问题二

问题二沿用问题一的设备结构，但信息边界发生改变：每天 0:00 制定全天计划时，当日真实负荷和光伏尚未实现，计划冻结后出现的供电缺口只能按正常电价的 5 倍紧

急购电。因此，附件中的当日真实曲线只能用于事后实现和评价，不能直接进入 0:00 的计划模型。

为描述全天计划面对的联合不确定性，先仅利用决策日前数据预测当日净负荷，再从历史整日预测残差中抽取场景，以保留 144 个时段间的共同偏移、峰谷变化和时序相关性。第一阶段的正常购电及储能充放电计划对所有场景相同；实际场景实现后，第二阶段仅允许紧急购电和弃电随场景变化，由题目的先后顺序形成严格两阶段随机 MILP。全年求解时按自然日期传递 SOC，并采用 48h 延伸优化、只执行前 24h 的方式估计日末储能价值，避免把储能在每天 24:00 无代价耗空。正式结果用样本外预测误差、紧急购电量、跨日 SOC 连续性和费用重算共同检验。

2.3 问题三

问题三在问题二基础上，于每日 0:00、6:00、12:00 和 18:00 依次发布新的光伏预测。每次信息更新时，过去时段的购电、充放电、费用和由此形成的当前 SOC 均已成为事实，不能回写；只有尚未执行时段可以调整。因此，模型必须显式保存当前 SOC、已执行决策和上一版仍有效的未来计划。

在每个更新时点，模型以当前 SOC 为新初值，用最新光伏预测替换剩余时域预测；负荷则只根据该时点之前已经观测到的负荷前缀进行因果偏差校正。随后重新优化未执行部分，仅执行到下一预报时点，再重复“观测—更新—优化—执行”。这一过程构成确定性滚动 MILP-MPC。费用账本不按最终计划一次性重算，而是将每一版新计划与更新前的有效计划比较，分别记录增购费用、减购退款，并在实际供电不足时计入紧急购电费用，从而保证计划变更与题目结算规则逐笔对应。最后通过 0 时、0/6 时、0/6/12 时和四次更新四组消融实验，评价新增预报的边际经济价值。

2.4 问题四

问题四进一步把事前已知的分时电价改为实时价格。附件 4 给出的是全年最终实现路径，但在正式决策时，未来价格与未来负荷、光伏一样不可见。若直接使用当天未来真实价格优化，将产生未来信息泄漏。因此，每个 0:00 计划和日内更新时点都设置历史数据截断，只允许用该时点以前的价格训练、选择并更新预测模型。

得到因果价格预测后，将其分别嵌入问题二的日前两阶段模型和问题三的日内滚动模型，形成 Q4-2 与 Q4-3 两种可执行策略：计划按决策时可获得的预测价格制定，最终费用按实际实时价格结算。为分离不同不确定性的影响，另设两个评价基准：完全价格信息基准只放宽未来价格信息，用于度量价格预测损失；全外生信息 oracle 同时给出价格、负荷和光伏的真实路径并放宽整数约束，只用于构造不可执行的成本下界。由此可以区分“现实策略成本”“价格未知的代价”和“全部未来信息的潜在价值”。

2.5 四问的递进关系与闭环

问题	决策时可用信息	允许调整范围	结算规则	主模型	核心检验
Q1	典型日价格、负荷、光伏全部已知	一次确定全日方案	正常购电价	确定性 MILP	无储能基线、LP 下界、约束残差

Q2	0:00 以前历史与当日预测场景	全天计划冻结	正常购电与 5 倍紧急购电	两阶段随机 MILP	样本外误差、跨日 SOC、追索费用
Q3	历史信息、已实现前缀及最新光伏预报	仅调整未执行时段	增购、减购退款与紧急购电	滚动 MILP-MPC	更新频率消融、逐版费用账本
Q4	上述信息及当前以前的实时价格	分别继承 Q2、Q3 规则	按实际实时价格结算	因果价格预测 + Q2/Q3	完全价格信息基准、全信息下界

四问由“完全信息—日前不确定—日内信息滚动—价格亦未知”逐层推进。上一问的物理模型和状态传递在下一问中保留，新增题目条件只改变信息集、允许调整的决策和费用函数。每次求解后，实际负荷、光伏和价格进入结算，并将期末 SOC 传给下一决策时点，从而形成闭合的动态决策链。总体过程如图 2-1 所示。综合以上分析，本文的总体求解过程如图 2-1 所示。

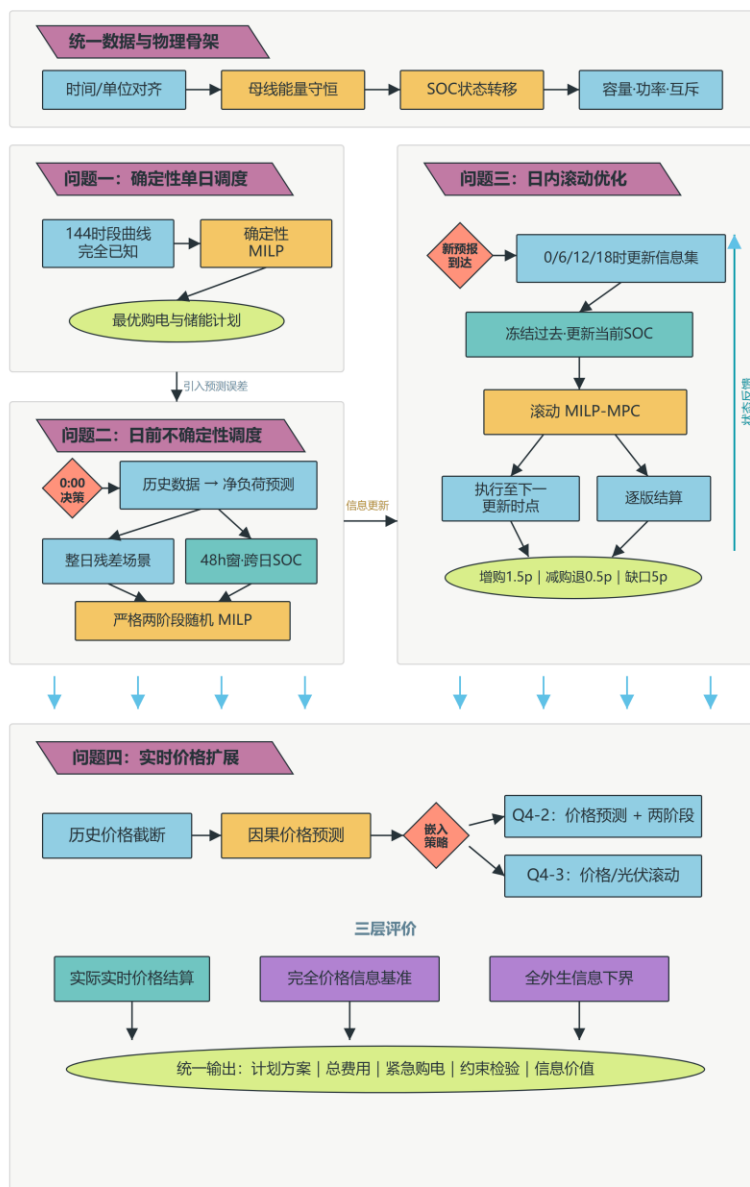


图 2-1 文章总体思路图

三、模型假设

3.1 基本假设

1. 时段平稳性假设：假设附件给出的负荷、光伏与电价数值代表其所属 10 分钟时段内的平均水平，即时段内功率取常值。附件的时间分辨率即为 10 分钟，在该尺度内采用分段常值既与数据口径一致，也能直接完成功率与电量的换算，从而把全天统一为 144 个可直接相加、相减的电量序列。

2. 能量流完整性假设：假设光伏既可直接供给负荷，也可用于储能充电，且不人为规定光伏、储能与外网的固定供电次序。实际调度取决于当期供需、储能状态与未来电价，固定优先级会割裂跨时段决策，因此模型只以母线功率平衡约束全部能量去向，供电次序由全时域费用最小化内生决定。

3. 交易边界封闭性假设：假设微网不具备反向上网通道与售电收益，富余光伏只能用于充电或弃置，弃光收益记为 0。题目未给出反向交易机制，该处理可避免将题目之外的收益来源计入目标函数，使优化结果只反映真实的购电支出。

4. 效率计量一致性假设：假设充电量与放电量均定义在交流母线侧，充、放电效率分别取题目给定的 90%。统一计量位置后，母线平衡与储电量递推之间不会重复计算损耗，设备损耗得以显式归集到储能状态转移关系之中。

5. 跨日状态连续性假设：假设储能是连续运行的设备，除题目另有规定外不在每日零时人为复位。这具体表现为：（1）问题二至问题四的日末储电量直接作为次日初值；（2）问题一仅采用题目明确给出的日初与日末储电量相等条件。该假设使全年多日问题不能被错误地拆分为彼此独立的单日优化。

6. 信息因果性假设：假设预测模型、标准化参数、情景残差及模型选择只使用决策时刻以前的数据，未来实际值仅在计划冻结后进入结算与评价环节。该假设界定了各问中决策者实际可获得的信息集，是日前策略与滚动策略在现实中能够执行的前提，也是问题二至问题四区别于完全信息情形的根本依据。

四、符号说明

表 4-1 主要符号及其说明

符号	说明
时间与情景标号	
d	日期序号
t	时段序号, $t = 1, 2, \dots, 144$
Δt	时段长度, $\Delta t = 1/6$ (单位: h)
ω	情景编号
q_ω	情景概率
输入数据与派生电量	

符号	说明
$L_{d,t}$	小区负荷功率（单位：kW）
$P_{d,t}^{pv}$	光伏发电功率（单位：kW）
$p_{d,t}$	外网购电单价（单位：元/kWh）
$\ell_{d,t}$	时段负荷电量， $\ell_{d,t} = L_{d,t}\Delta t$ （单位：kWh）
$S_{d,t}$	时段光伏发电量， $s_{d,t} = P_{d,t}^{pv}\Delta t$ （单位：kWh）
$n_{d,t}$	净负荷电量，由时段负荷电量减去光伏发电量得到（单位：kWh）
$\eta_c、\eta_d$	储能充、放电效率，均取 0.9
决策变量与储能状态	
$G_{d,t}$	计划外网购电量（单位：kWh）
$C_{d,t}$	储能充电量（单位：kWh）
$D_{d,t}$	储能放电量（单位：kWh）
$R_{d,t}$	紧急购电量（单位：kWh）
$W_{d,t}$	弃电量（单位：kWh）
$E_{d,t}$	时段 t 起始时刻的储电量（单位：kWh）
$z_{d,t}$	充放电互斥状态变量， $z = 1$ 表示充电、 $z = 0$ 表示放电
$\Delta_{d,t}^+、\Delta_{d,t}^-$	购电计划的上调量与下调量（单位：kWh）
目标函数与费用指标	
C_1	问题一中全天的购电总费用（单位：元）
C_d^2	问题二中日前计划的目标费用期望（单位：元）
C_d^3	问题三中滚动调整后的日总费用（单位：元）

五、数据预处理与探索性分析

5.1 数据预处理

为确保后续建模的准确性与可靠性，并满足前述基本假设，需要对原始数据完成时段对齐、单位换算、缺失值识别与取值范围核查。各附件在时间标签方式、数据粒度与缺失形态上并不一致，因此预处理按以下步骤逐项执行：

- **时段划分与时间标签对齐：**记每日时段集合为 $\mathcal{T} = \{1, 2, \dots, 144\}$ ， $\Delta t = \frac{1}{6}h$ 。

原始附件依次给出“0:10, 0:20, ..., 0:00+1”共 144 项，附件 5 又把首项明确写为“0:10—0:20”。因此本文把源数据中的 0:10 解释为区间起点，对应 0:10-0:20；其余各项按同一规则顺延，末项对应次日 0:00-0:10。计算和结果回填始终采用同序号对应，不把序列循环平移。

• **功率与电量的单位换算：**负荷与光伏功率分别记为 $L_{d,t}$ 、 $P_{d,t}^{pv}$ ，对应的十分钟电量为 $\ell_{d,t} = L_{d,t}\Delta t$ ， $s_{d,t} = P_{d,t}^{pv}\Delta t$ 。统一换算为电量后，母线能量平衡与储电量递推可在同一量纲下直接相加、相减。

• **缺失值识别与取值范围核查：**对附件 1-4 逐表检查日期、时段、缺失值和取值范围。附件 1、附件 2 及附件 4 均无缺失值和负值；附件 3 有 1095 个日期空白，均来自同一日期四个预报时刻只在首行填写日期的版式设计，故在组内向下填充，而不作为观测缺失插补。原始功率观测经范围核查后不作主观删改，统一乘以 1/6 小时转换为每个 10 分钟时段的电量；附件 3 整点预测插值到 10 分钟网格后仅作非负裁剪。

5.2 数据探索性分析

本小节对负荷、光伏、净负荷与电价等关键量的量级、分布与相互关系进行探索性分析，所得结论为后续模型的数据使用方式、时间口径与滚动策略提供依据。

5.2.1 负荷与光伏的量级特征

统计表明，附件 1 的负荷位于 3309.39—5958.97kW 之间，而光伏最高可达 7612.32kW，且日间由零出力开始爬升，两者在时序上并不同步。负荷波动区间相对集中，属于必须被持续满足的刚性需求；光伏出力则在全量程内变化，只能作为间歇性来源参与供给。这一差异决定了微网的供电能力不能只依赖光伏，必须由外购电与储能放电共同兜底，因此模型不预先规定光伏、储能与外网的固定使用次序，而由全时域费用最小化决定。

5.2.2 净负荷的符号特征

净负荷由时段负荷电量减去同时段光伏电量得到，全年中有 30 个时段出现负值，说明部分日间时段存在可用于充电的光伏富余。该特征具有两重含义：其一，储能在这些时段拥有不依赖外购电的充电来源，跨时段转移电量具备现实可行性；其二，光伏出力仍可能超过负荷与充电功率上限之和，因此弃光量必须作为独立决策变量保留，而不能假定光伏总能被完全消纳。

5.2.3 电价的峰谷结构

分时电价为 0.3713—1.3952 元/kWh，日内存在明确的峰谷台阶，峰谷差使储能具有跨时段转移价值：低价时段充电、高价时段放电可以降低全天购电费用。这正是问题一至问题三中储能收益的来源，也说明目标函数须按各时段购电量与对应电价的乘积求和，而不能用平均电价近似。

5.2.4 典型日与全年数据的一致性

附件 2 覆盖 365 个连续日期，附件 1 的负荷和光伏曲线分别与附件 2 逐时刻的全年均值高度一致，因此附件 1 可作为全年典型日均值的代表。据此，问题一按单日 144 个时段一次性求解；问题二至问题四按自然日顺序滚动计算，并在相邻日之间连续传递储电量，而不把全年问题拆分为彼此独立的单日优化。

六、问题一的模型建立与求解

6.1 数据处理与参数设置

问题一采用附件 1 提供的单日电价、小区负荷和光伏发电功率预测数据，并按照 10min 的调度间隔进行统一。附件中的功率数据单位为 kW，购电量和储能充放电量单位为 kWh，因此将各时段功率乘以时间间隔 $\Delta t = \frac{1}{6}$ h，得到对应时段的电量。数据处理过程中检查时间序列是否完整、是否存在重复或缺失记录，并将“0:00+1”统一视为当天 24:00。储能设备的额定容量、储电量上下限、最大充放电功率及充放电效率均按照附录 1 给定参数设置，初始储电量按照题目规定取值。经处理后的数据作为问题一确定性日内调度模型的输入。

6.2 模型建立

6.2.1 时段电量平衡

在时段 t 开始时，储电量已由上一时段的运行状态确定。该时段的光伏发电量、计划购电量和储能放电量共同向交流母线供电，居民负荷用电量和储能充电量从交流母线取电；当光伏发电量超过负荷需求及储能可执行充电量时，剩余部分计为弃光量。因此，各时段均满足以下电量平衡关系：

$$G_{d,t} + R_{d,t} + S_{d,t} + D_{d,t} = \ell_{d,t} + C_{d,t} + W_{d,t} \quad (1)$$

式（1）完整描述了该时段的电量来源及去向，并未预先规定光伏、储能和外网的供电顺序。由于光伏发电的边际成本为 0，且题目未设置售电收益，优化结果通常会尽可能利用光伏发电功率；但具体的充电、放电和购电组合仍需综合考虑储能容量、电价水平及后续时段的负荷需求，通过全时域费用最小化确定。式中的弃光量表示在统一转换为 10min 时段电量后，光伏发电量超过当期负荷用电量和储能可执行充电量的剩余部分。

6.2.2 储能状态、边界与充放电互斥

充电量和放电量均定义在交流母线侧。储能充电时，交流母线输入 1kWh 电量后，电池内部实际增加 0.9kWh；储能放电时，若向交流母线输出 1kWh 电量，则电池内部需要减少 1/0.9kWh。由此，当前储电量、充电量和放电量共同决定下一时段的储电量：该状态递推方式与储能经济调度中的 SoC 建模原理一致^[1]。

$$E_{d,t+1} = E_{d,t} + 0.9C_{d,t} - \frac{D_{d,t}}{0.9} \quad (2)$$

储能容量和充放电功率必须满足以下边界约束：

$$1200 \leq E_{d,t} \leq 10800 \quad (3)$$

$$0 \leq C_{d,t} \leq \frac{5000}{6} z_{d,t}, 0 \leq D_{d,t} \leq \frac{5000}{6} (1 - z_{d,t}), z_{d,t} \in \{0,1\} \quad (4)$$

充放电互斥约束用于描述单套储能设备的实际运行状态。若允许同一时段同时充电和放电，模型可能通过人为制造充放电损耗来减少弃光量，这种运行方式不具有物理意义。因此，本文设置二元状态变量，限制储能在同一时段只能处于充电、放电或静止状态。去除二元状态约束的连续模型仅用于计算费用下界和开展对照分析。

6.2.3 初始储电量条件

题目给出了储能设备在 2025 年 1 月 1 日 0:00 的初始储电量为6000kWh，本文将其作为问题一的初始状态。根据问题一储能设备在 0:00 和 24:00 的储电量相同的要求，设置首末储电量相等，从而保证调度周期闭合，并避免通过消耗期末储电量人为降低当日购电费用。

6.2.4 目标函数与完整模型

问题一仅研究题目给定的单日调度，以下省略日期下标 d。144 个时段的电价、小区负荷和光伏发电功率预测数据均已知，不涉及计划偏差和紧急购电。模型的目标是在满足各时段供电需求及储能运行约束的条件下，确定各时段的外网购电量、储能充电量和放电量，使全天购电费用最小。因此，目标函数为各时段购电量与对应电价乘积之和。

minC1 = \sum_{t=1}^{144} pd_t G_t (5)

模型满足式（1）-（4），并满足相应的变量取值范围及首末储电量相等条件。

求解时，将 144 个时段的决策变量和约束一次性联立。各时段的电量平衡约束描述当期供需关系，储电量递推约束连接相邻时段，首末储电量相等条件则使整个调度周期闭合。由于目标函数和连续约束均为线性形式，仅需通过二元变量表示充放电互斥关系，即可将问题表述为混合整数线性规划模型，并利用 HiGHS 求解器进行求解；当求解器返回最优状态且相对最优间隙满足设定精度时，所得方案即为该模型的全局最优解。

E1 = E145 = 6000 (6)

6.3 模型求解与计算结果

将 144 个时段的购电量、充电量、放电量、弃光量、储电量及充放电状态按时间顺序组成决策向量，并将式（1）—（6）转化为稀疏线性约束，采用 HiGHS 混合整数规划求解器，通过分支定界框架求解该模型。求解完成后，按照原始时间顺序还原各项能量流，并逐时复核电量平衡、储能状态递推及充放电互斥约束。同时，分别求解无储能基线方案和连续松弛模型，用于评估储能设备的经济效益及整数互斥约束对最优结果的影响。

为判断储能设备是否能够降低全天购电费用，并检验整数互斥约束是否改变最优值，表 6-1 在统一统计口径下对比主模型、连续松弛模型和无储能基线方案。

表 6-1 问题一全天调度指标与对照

指标	MILP 主模型	连续松弛	无储能方案
日购电费用/元	35126.95	35126.95	48052.05
日购电量/kWh	59482.70	59482.70	—
充电量/kWh	20740.67	20740.67	0
放电量/kWh	16799.94	16799.94	0
弃电量/kWh	0.00	0.00	0
同时充放电时段数	0	0	0

由表 6-1 可知，主模型的日购电费用为 35126.95 元，较无储能方案的 48052.05 元减少 12925.10 元，降幅为 26.90%。连续松弛模型与混合整数模型的购电费用、充放电

量及弃光量均相同，说明在本组数据下，连续松弛模型的最优解已自然满足充放电互斥条件；二元变量虽然未改变最优目标值，但为模型显式施加了物理运行约束。

在确认储能总体经济效益的基础上，进一步考察模型在题目指定时段的购电分配结果。表 6-2 列出了六个指定 10min 时段的购电量及全天汇总结果。

表 6-2 指定时段购电量及全天汇总

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量	59482.70	全天购电费	35126.95		

由表 6-2 可知，10:00、14:00 和 20:00 的购电量为 0，而 18:00 的购电量达到 636.98kWh。各时段购电量的差异是光伏发电功率、电价水平、储能荷电状态及 5000kW 充放电功率上限共同作用的结果，并非由预先设定的能源使用顺序决定。

为说明上述购电差异与储能跨时段调节之间的关系，表 6-3 进一步汇总各四小时时段的充、放电量，并核对调度周期的首末储电量。

表 6-3 储能分段充/放电量及首末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00-4:00	4500.00	0.00	4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量	6000.00	24:00 储电量			6000.00

由表 6-3 可知，储能主要在 0:00—4:00 以及光伏发电量较为充足的 8:00—16:00 时段充电，并在 4:00—8:00 和 16:00—20:00 时段集中放电。24:00 储电量恢复至 6000.00kWh，与 0:00 的初始储电量一致，表明费用节约并非通过消耗日末库存实现。

在表格给出精确数值的基础上，图 6-1 将供需功率、储能充放电动作和荷电状态置于同一时间轴上，以检验调度结果是否与光伏发电功率及电价变化相匹配。

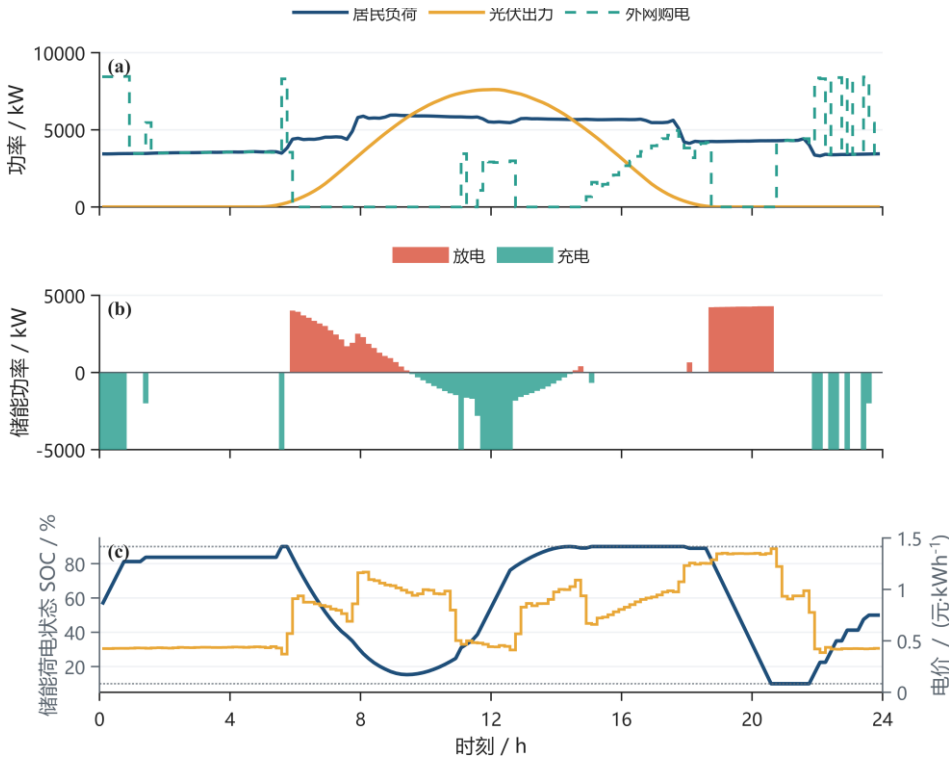


图 6-1 典型日功率、储能动作与 SOC 轨迹

图 6-1(a)表明, 当光伏发电功率能够覆盖部分小区负荷时, 外网购电量相应减少; 图 6-1(b)显示, 储能充电主要发生在电价较低或光伏发电量较为充足的时段, 放电则主要集中于电价较高且净负荷较大的时段; 图 6-1(c)表明, 各时段储电量始终处于 1200—10800kWh 的允许范围内, 对应荷电状态 SOC 处于 10%—90%之间, 并在日末恢复至初始值。三幅子图分别从供需关系、储能响应和状态约束三个方面验证了调度方案的合理性。

6.4 结果分析与检验

综合上述结果, 问题一的最优策略并非预先规定光伏、储能和外网的固定供电顺序, 而是在逐时电量平衡和储电量递推约束下, 通过全天购电费用最小化自然形成削峰填谷效果。储能将低价或光伏发电量较为充足时段的电量转移至高价或供电不足时段, 使日购电费用下降 26.90%; 同时, 日末储电量与初始值相同, 说明该节省效果未以透支周期末储能为代价。

从可行性检验结果看, 各时段母线电量平衡和储电量递推的最大残差均小于 10^{-12} kWh, 储电量始终位于 1200—10800kWh 范围内, 且不存在同一时段同时充电和放电的情况。连续松弛模型与混合整数模型取得相同目标值, 进一步说明在本组数据下, 充放电互斥约束未导致目标值上升, 同时保证了所得方案的物理可执行性。因此, 问题一的费用结果、储能运行机理和约束检验结果彼此一致。

七、问题二的模型建立与求解

7.1 数据处理与情景构造

将小区负荷减去光伏发电功率得到净负荷后, 预测对象不再是彼此独立的 144 个时点, 而是具有共同变化特征的全天净负荷轨迹。为此, 本文首先利用决策日前的历史数据, 对比 1~7 日季节加权基线模型与包含日历特征、滞后项的岭回归模型。对于每个月的首日, 采用此前最近 14 个完整日进行滚动验证, 并依据加权绝对百分比误差选择预测效果较稳定的模型。随后, 保存历史整日预测残差向量:

$$n_{d,t}^{\omega} = \hat{n}_{d,t} + e_{j_{\omega},t} \quad (7)$$

从历史残差库中按整日抽样, 以保留 144 个时段之间的相关变化, 避免因逐时独立抽样而组合出实际中不易出现的净负荷曲线。各历史日按照经验频率赋予相应概率, 并在 10、20 和 30 个候选情景中选择分布统计结果趋于稳定的最小情景数, 以兼顾情景代表性和全年求解效率。

7.2 两阶段模型建立

$$J_{d,0}^{(2)} = \{E_{d,1}, p_{1:144}, \text{日历特征}, (L_{j,:}, P_{j,:}^{pv})_{j < d}\} \quad (8)$$

对于同一决策日的不同实际情景, 0:00 时确定的计划购电量、充电量和放电量应保持一致; 实际净负荷揭示后, 仅根据供电缺口计算紧急购电量, 并记录相应的弃光量。由于问题二规定计划购电量在日前确定, 除紧急购电外的其他时段按照计划购电量结算, 因此模型将决策变量划分为日前计划和实际运行后的情景响应两部分。该处

理方式既符合题目给出的信息获取顺序和费用结算规则，也能够反映光伏发电功率与小区负荷不确定性对运行费用的影响[7][8][9]。

$$G_{d,t} + R_{d,t}^{\omega} + D_{d,t} - C_{d,t} - W_{d,t}^{\omega} = n_{d,t}^{\omega} \quad (9)$$

第二阶段不重新安排储能充放电，是为了遵循题目对事后补救措施的限定，同时明确区分问题二与问题三：问题二中的日前计划在制定后保持不变，问题三则利用题目规定的新光伏发电功率预报，对尚未执行的时段进行调整。

问题二和问题三均按照自然日期连续运行，不设置人为的每日复位；问题四分别在波动电价条件下重新执行问题二和问题三的调度过程，并沿用相应的跨日储能状态。且满足：

$$E_{d+1,1} = E_{d,145} \quad (10)$$

问题二至问题四不设置人为的每日复位，上一日 24:00 的储电量直接作为下一日 0:00 的初始储电量。运行从 1 月 1 日储电量 6000kWh 的状态开始，按照自然日期连续推进，1 月 31 日 24:00 的储电量作为 2 月 1 日 0:00 的实际初始状态。为减弱独立逐日优化产生的末端效应，本文在每日 0:00 求解时引入 48h 延伸窗口，并仅执行其中前 24h 的计划，以避免在调度日末无约束地释放储能；延伸时段用于估计日末储电量的后续价值，不作为当日实际执行方案。

对于每个候选实际情景，正常购电费用由同一份日前计划确定，紧急购电费用则由该情景下的供电缺口决定。按照情景概率进行加权后，在 0:00 选择使正常购电费用与期望紧急购电费用之和最小的计划：

$$\min C_d^{(2)} = \sum_t p_t G_{d,t} + \sum_{\omega} q_{d,\omega} \sum_t 5 p_t R_{d,t}^{\omega} \quad (11)$$

日前计划确定后，根据当天实际小区负荷和光伏发电功率逐时结算。当光伏发电功率、计划购电量及既定储能放电量仍不足以满足小区负荷时，不足部分计为紧急购电量；当光伏发电功率超过小区负荷和储能可执行充电量时，超出部分计为弃光量。两者分别由下式计算：

$$R_{d,t}^{act} = \max\{n_{d,t}^{act} + C_{d,t} - G_{d,t} - D_{d,t}, 0\} \quad (12)$$

$$W_{d,t}^{act} = \max\{G_{d,t} + D_{d,t} - C_{d,t} - n_{d,t}^{act}, 0\} \quad (13)$$

7.3 模型求解与计算结果

全年运行按照自然日期顺序递归求解。对于每个决策日，首先利用截至前一日的历史数据生成净负荷基线和历史残差场景，再求解包含 48h 延伸窗口的两阶段模型，实际仅执行并结算前 24h 的决策结果，并将当日 24:00 的储电量传递至下一日 0:00。程序分别累计计划购电费用、紧急购电费用和紧急购电量，同时针对题目指定日期输出各时段的计划购电量、实际供电缺口及储能状态。

在 2025 年 2 月 1 日至 12 月 31 日的正式统计期间，计划购电费用为 13750414.64 元，紧急购电费用为 1507608.25 元，总费用为 15258022.89 元；紧急购电量为 404258.73kWh，预测加权绝对百分比误差为 10.95%。下文分别从指定日期结果、代表日供需偏差、月度费用结构和跨日储能状态连续性四个方面展示计算结果。

首先，表 7-1 给出了题目指定日期及指定时段的日前购电安排，并列出了各代表日的全天购电量和购电费用。

表 7-1 指定日期、指定时段的购电量及全天汇总

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量/kWh	全天购电费/元
2025-03-20	0.00	805.09	0.00	458.78	756.58	0.00	73287.68	45433.65
2025-06-21	0.00	0.00	0.00	125.03	446.17	0.00	44714.68	24923.97
2025-09-23	0.00	716.69	0.00	382.61	836.37	9.31	72727.19	44944.55
2025-12-21	0.00	1096.15	0.00	853.43	712.22	0.00	98305.49	63043.85

由表 7-1 可知，四个代表日的购电规模随季节净负荷变化而改变。其中，6 月 21 日的全天购电量相对较低，而 3 月 20 日和 12 月 21 日的全天购电量相对较高。表中各指定时点的购电量均属于 0:00 时确定的日前计划，不包含实际运行过程中产生的紧急购电量。

由于日前购电计划需要与储能运行状态共同满足负荷需求，表 7-2 进一步按照六个四小时时段汇总各代表日的充电量和放电量，并列出每日 0:00 与 24:00 的储电量。

表 7-2 指定日期储能分段充/放电量（单元格为充电量/放电量，kWh）

日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时	储电量(0 时/24 时)
2025-03-20	1666.67/0.00	1666.67/43.52.03	5833.33/47.31.59	5833.33/10.41.37	0.00/5645.40	4335.30/29.94.60	8550.00/5101.77
2025-06-21	462.26/0.00	833.33/247.2.90	5015.46/0.00	3285.30/38.71	0.00/5867.15	8166.67/27.72.85	4953.95/8550.00
2025-09-23	1666.67/0.00	833.33/619.8.05	6282.75/24.41.95	5031.89/52.4.86	0.00/5809.57	8166.67/28.30.43	8550.00/8550.00
2025-12-21	1666.67/0.00	1666.67/15.08.33	5833.33/78.06.67	8333.33/28.35.00	0.00/5733.07	8166.67/29.06.93	8550.00/8550.00

由表 7-2 可知，不同日期的储能充放电安排会随预测净负荷变化而调整。各日 24:00 的储电量由当天实际执行结果决定，并直接传递至下一日 0:00；首末储电量不作相等处理，从而避免将全年调度错误地拆分为相互独立的单日优化问题。

日前计划确定后，仅当实际供电量不足以满足小区负荷时才产生紧急购电。表 7-3 列出了四个指定日期中发生供电缺口的时间区间及对应的紧急购电量。

表 7-3 指定日期紧急购电

2025-03-20 紧急购电时间段		购电量/kWh
0:30-0:40		3.63
1:10-1:50		71.80
3:20-3:40		47.00
3:50-4:00		8.59
8:20-8:40		63.01
15:50-16:20		81.81
18:50-19:10		38.84
20:20-20:30		2.60
21:20-21:50		12.20
23:50-0:00+1		7.34
2025-06-21 紧急购电时间段		购电量/kWh
0:30-0:40		12.70
1:30-1:40		21.30
2:00-2:10		23.40
3:00-3:40		30.30
4:10-4:30		34.64

2025-03-20 紧急购电时间段	购电量/kWh
5:50-6:30	47.48
21:30-21:40	26.72
2025-09-23 紧急购电时间段	购电量/kWh
0:50-1:00	14.23
2:20-2:30	7.79
5:00-5:20	13.89
6:20-7:10	53.79
9:40-10:30	276.22
15:00-15:30	60.74
16:00-17:00	440.20
17:10-17:30	35.54
18:20-18:30	31.25
20:20-20:30	35.09
21:00-21:10	33.06
21:30-21:40	5.55
22:30-22:40	12.58
2025-12-21 紧急购电时间段	购电量/kWh
0:20-0:30	19.87
1:20-2:00	84.78
2:10-2:30	16.41
3:10-3:30	55.57
3:40-4:00	158.15
5:10-5:30	8.36
6:00-6:10	31.39
6:30-6:40	30.04
7:20-7:30	4.86
7:50-8:10	60.03
11:10-11:40	149.05
15:30-15:40	46.14
17:00-17:10	15.34
17:20-17:50	39.11
18:00-19:10	160.64
20:00-20:10	44.42
20:40-20:50	15.34
21:20-21:30	1.37
23:00-23:20	33.82
23:40-0:00+1	62.62

由表 7-3 可知，紧急购电主要集中在若干连续的短时段，表明较高的紧急购电费用主要由局部预测偏差造成，而非全天供电能力不足。该结果也说明，在实际数据揭示后，模型仅对未被既定供电方案覆盖的负荷缺口进行紧急购电结算，并未重新安排当日储能充放电计划。

为进一步说明紧急购电的产生原因，图 7-1 将四个代表日的日前预测净负荷、实际净负荷及紧急购电时段绘制在同一图中。

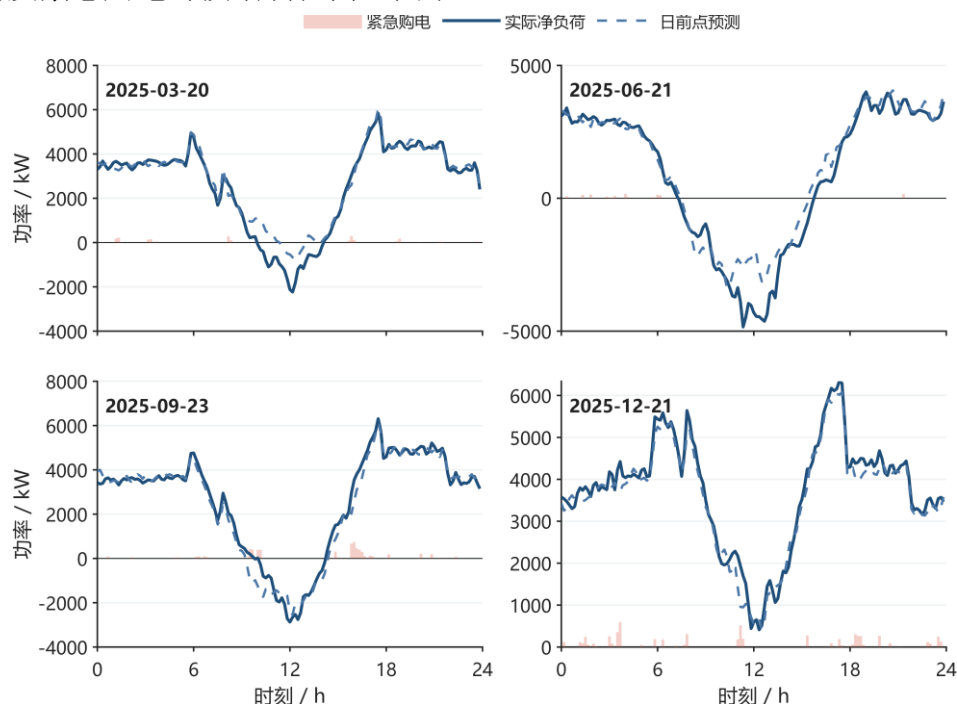


图 7-1 四个代表日的日前预测、实际净负荷与紧急购电

图 7-1 表明，当实际净负荷高于日前预测，且既定计划购电量与储能放电量不足以覆盖该差额时，便会产生紧急购电。6 月 21 日的预测偏差相对较小，全天紧急购电量为 196.54kWh；其余代表日的紧急购电时段主要与净负荷被低估的峰值区间相对应，符合问题二的两阶段结算规则。

代表日结果只能反映局部运行特征，因此图 7-2 进一步汇总正式统计期间各月份的计划购电费用、紧急购电费用和紧急购电量，用于分析季节性差异。

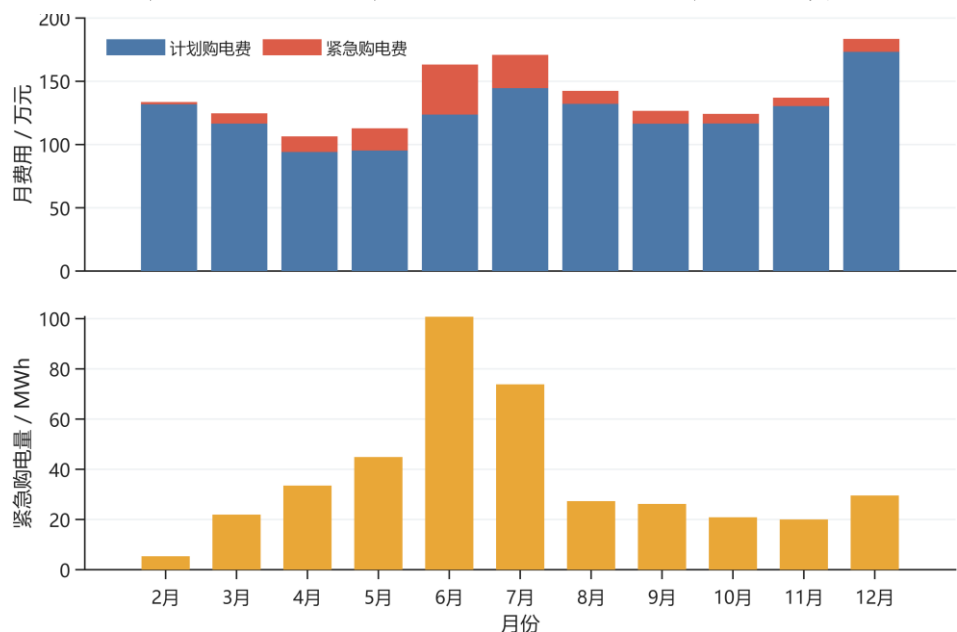


图 7-2 正式期月度费用分解与紧急购电量

图 7-2 显示，计划购电费用始终是月度总费用的主要组成部分，紧急购电费用占全年总费用的 9.88%。6 月和 7 月的紧急购电量相对较高，说明光伏发电功率变化及净负荷预测误差具有一定的季节性，并会影响实际供电缺口的规模，但未改变计划购电在全年供能结构中的主体地位。

全年费用结果只有在储能状态连续且始终满足边界约束的情况下才具有可信度。图 7-3 给出了正式统计期间每天的日初储电量、日末储电量以及日内最小和最大储电量。



图 7-3 正式期跨日 SOC 连续性及其日内运行范围

图 7-3 表明，正式统计期间的日内最小储电量为 1997.08kWh，最大储电量为 8550.00kWh，均处于题目规定的 1200~10800kWh 范围内。每天 24:00 的储电量与下一日 0:00 的初始储电量逐日衔接，最大连接残差为 0kWh。因此，全年费用结果并非通过每日重置储能状态或割裂跨日储能关系获得。

7.4 结果分析与检验

问题二的全年计算结果表明，严格的日前计划能够承担大部分供能任务，但预测误差会在计划冻结后转化为高价紧急购电。正式统计期间的计划购电费用为 13750414.64 元，紧急购电费用为 1507608.25 元，占总费用的 9.88%；紧急购电量为 404258.73kWh，主要集中在实际净负荷超过日前供电覆盖范围的局部时段，符合题目规定的事后补救方式。

模型在运行过程中按照自然日期连续推进，日末储电量直接传递至下一日，全年跨日连接残差为 0kWh。同时，预测样本、历史残差场景及模型参数均仅依据决策日前可获得的数据生成，未使用决策日之后的实际信息。结合母线电量平衡、储电量边界和预测误差检验结果，可以确认全年费用差异主要来源于信息不完全条件下的预测偏差，而不是每日重置储能状态或使用未来数据造成的结果。

八、问题三的模型建立与求解

问题三与问题二的差别不在物理模型，而在信息到达过程：题目在每天 0:00、6:00、12:00 和 18:00 依次发布新的光伏预报，但已执行时段的购电、充放电和费用已经成为事实，不能回写调整。因此本问按三步展开：首先明确每个滚动节点的预报处

理方式与状态继承规则，保证决策不越过信息边界；其次以当前实测储电量为初值重建剩余时域的确定性混合整数线性规划，并定义新计划相对上一版有效计划的增购与减购量，形成非对称调整结算；最后通过更新频率的消融对照与逐版费用账本，检验新增预报是否具有正的边际经济价值。

8.1 预测数据处理与状态更新

本小节明确每次滚动求解的信息边界与状态继承规则，为后续滚动优化提供一致的起点。

- **光伏预报的处理：**以发布时刻的实测值为 0 小时锚点，将附件 3 的整点光伏预报线性插值到 10 分钟网格，使预报与负荷、电价落在同一时间轴上。这种将功率预测与调度时标对齐的处理可使预报误差直接进入后续决策^[3]。

- **负荷的因果修正：**以历史同星期曲线为基线，结合当天已经实现的负荷偏差进行稳健修正，不使用发布时刻之后的数据。

- **状态与已执行量的继承：**从当前实测储电量出发，只对未执行时段重新求最小费用计划，已执行时段的购电、充放电与费用保持不变。

上述处理只使用发布时刻已经获得的信息，因此不会把后续实际值泄漏给当前决策。

8.2 滚动优化模型建立

本小节在问题一的物理骨架上建立滚动优化模型，并给出计划调整量与全年费用的结算口径。

8.2.1 滚动求解的继承与冻结

每次滚动求解均继承问题一的能量平衡、储能状态和互斥约束，并以当前实测储电量作为新初值，仅对未执行时段重新求解。该处理与微网日内调度和滚动时域控制的基本思想一致[2][4][5][6]。

8.2.2 非对称调整量

设调整前仍有效的购电计划为原计划，新优化得到的未执行时段购电量为新计划；两者的正差计为增购，负差的绝对值计为减购，由此得到非对称调整费用。

记 $\bar{G}_{d,t}^{(h-)}$ 为时刻 h 调整前的有效计划， $G_{d,t}^{(h)}$ 为新计划。定义

$$\Delta G_{d,t}^{+,h} = \max\{G_{d,t}^{(h)} - \bar{G}_{d,t}^{(h-)}, 0\}, \Delta G_{d,t}^{-,h} = \max\{\bar{G}_{d,t}^{(h-)} - G_{d,t}^{(h)}, 0\} \quad (14)$$

8.2.3 调整费用与总费用构成

每次调整相对上一版有效计划逐次结算：

$$C_d^{\text{adj},h} = \sum_{t \geq h} (1.5p_t \Delta G_{d,t}^{+,h} - 0.5p_t \Delta G_{d,t}^{-,h}) \quad (15)$$

减购项前的负号表示从已经计入的原计划费用中退回减购量正常价格的 50%，不是另加 50% 罚款。实际总费用为

$$C_d^{(3)} = \sum_t p_t G_{d,t}^{(0)} + \sum_{h \in \{6,12,18\}} C_d^{\text{adj},h} + \sum_t 5p_t R_{d,t}^{\text{act}} \quad (16)$$

8.3 模型求解与计算结果

本小节依次从更新频率的消融对照、滚动预测的修正过程、购电与储能轨迹的调整以及残余紧急购电四个层面，检验滚动策略的实际效果。

8.3.1 更新频率的消融对照

对每个运行日依次设置 0 时、6 时、12 时和 18 时四个决策节点。每到一个节点，程序读取当前储电量和最新预测，冻结此前已经执行的变量，只对剩余时段重新求解；新旧有效购电计划的差额随即进入调整账本。为检验更新频率的价值，分别计算仅 0 时、增加 6 时、增加 12 时以及四次更新的完整年度结果。

全年计算得到：仅在 0:00 制定计划的总费用为 16391391.76 元，采用 0:00、6:00、12:00 和 18:00 四次更新后的总费用为 15731687.04 元，调整净费用为 499519.56 元，紧急购电量为 748796.98kWh。接下来先用消融对照判断更新是否值得，再展示预测修正过程和指定日期的最终执行结果。

是否采用全部更新时刻必须比较新增信息带来的应急费用下降与调整成本后再作出判断。表 8-1 依次加入 6:00、12:00 和 18:00 更新，形成可比的年度消融实验。

表 8-1 不同预报更新频率的全年代价对照

更新时刻集合	总费用/元	调整费用/元	紧急购电量/kWh	求解时间/s
仅 0:00	16391391.76	0	1040382.80	76.04
0:00、6:00	15926257.20	322397.90	841482.17	119.31
0:00、6:00、12:00	15753473.50	436298.01	767041.40	147.07
0:00、6:00、12:00、18:00	15731687.04	499519.56	748796.98	133.33

表 8-1 显示，更新时刻由仅 0:00 更新增加到更新四次后，总费用从 16391391.76 元降至 15731687.04 元，节省 659704.73 元；紧急购电量从 1040382.80kWh 降至 748796.98kWh，下降 28.03%。尽管调整费用上升到 499519.56 元，但紧急购电损失的减少更大，因此完整更新在本组数据上仍有净收益。

为同时观察总费用、调整费用和紧急购电量的变化方向，图 8-1 将表 8-1 的核心指标转化为同一组视觉对照。

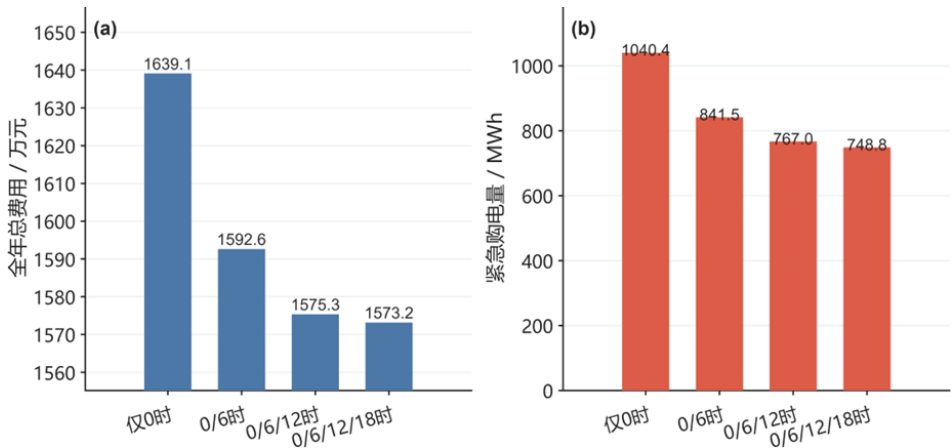


图 8-1 不同日内更新频率的费用与紧急购电量对比

图 8-1 直观显示，总费用与紧急购电量随更新次数增加而单调下降，但边际收益逐步减小：加入 18:00 更新后总费用仅再下降 21786.47 元，而调整费用继续增加。由此可见，第四次更新虽仍有效，却已接近收益递减区间。

8.3.2 滚动预测的修正过程

年度消融说明更新有效，但还需检验它如何改变日内决策。图 8-2 展示四个指定日期中 0:00、6:00、12:00 和 18:00 预测对剩余净负荷轨迹的逐步修正。

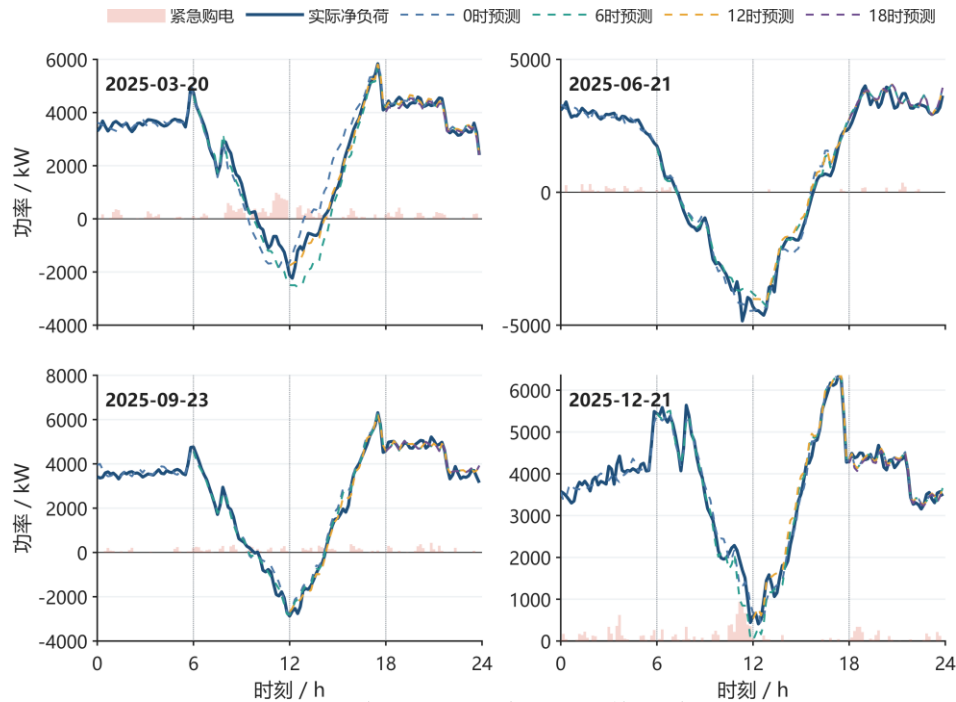


图 8-2 四个指定日的滚动预测修正效果

图 8-2 中，每条更新曲线只从其发布时间向后延伸，发布时间之前的已执行区间不被改写；后续预测逐步贴近实际轨迹，使剩余购电计划能及时修正。该图同时证明滚动策略遵守因果信息边界，而不是利用整日实际值进行事后重算。

8.3.3 购电计划与储能轨迹的调整

预测修正最终会落实为购电计划变化。表 8-2 将指定时间点的 0 时计划量与最后一次更新后的有效量并列，便于核对调整方向和幅度。

表 8-2 指定日期、指定时段的 0 时计划量/最终有效量及全天汇总

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量/kWh	全天购电费/元
2025-03-20	0.00/0.00	538.61/44 1.63	0.00/0.00	587.79/44 1.17	723.27/72 5.78	0.00/0.00	64162.72	55675.48
2025-06-21	0.00/0.00	0.00/0.00	0.00/0.00	112.35/69 .11	417.02/43 1.81	0.00/0.00	34242.49	22452.33
2025-09-23	0.00/0.00	382.08/36 9.56	0.00/0.00	494.89/43 1.52	805.85/80 5.98	0.00/0.00	66693.08	50514.40
2025-12-21	0.00/0.00	981.28/84 7.42	0.00/0.00	796.14/81 5.36	717.22/71 8.72	0.00/0.00	93735.09	68375.78

表 8-2 显示，不同时段的有效购电量既可能上调也可能下调，且变化集中在更新时刻之后。结果说明模型没有把“更新”机械理解为增加购电，而是依据新预测重新平衡正常购电、储能和潜在紧急缺口。

购电计划变化必须与真实可执行的储能轨迹一致，因此表 8-3 给出相同日期的分段充放电量和首末荷电状态。

表 8-3 指定日期储能分段充/放电量（单元格为充电量/放电量，kWh）

日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时	SOC(0 时/24 时)
----	-------	-------	--------	---------	---------	---------	---------------

日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时	SOC(0 时/24 时)
2025-03-20	1666.67/0.0 0	833.33/642 9.36	4328.27/22 10.64	6659.67/26 0.23	0.00/5806.9 7	8166.67/28 33.03	8550.00/8550.00
2025-06-21	311.23/0.00	833.33/177 2.62	1575.61/0.0 0	9091.06/0.0 0	33.57/6089. 95	1355.08/25 77.25	2139.47/2419.58
2025-09-23	1666.67/0.0 0	833.33/686 0.17	5433.73/17 79.83	5633.97/32 4.84	0.00/5654.8 0	8166.67/29 85.20	8550.00/8550.00
2025-12-21	1666.67/0.0 0	1661.40/15 65.11	5833.33/74 69.66	7642.90/25 51.71	0.00/5739.7 2	8166.67/29 00.28	8550.00/8550.00

表 8-3 表明，各日储能在新预报到达后重新分配剩余时段的充放电，但始终从当时实测状态继续递推。首末状态差异反映跨日能量转移，而非约束遗漏；逐时检查确认容量、功率和互斥条件均满足。

8.3.4 残余紧急购电的分布

为检验更新后仍未消除的风险，表 8-4 列出指定日期最终需要紧急购电的区间与电量。

表 8-4 指定日期紧急购电

2025-03-20 紧急购电时间段	购电量/kWh
0:30-0:40	28.15
1:00-1:50	188.73
2:40-2:50	3.74
3:10-4:10	146.15
5:00-5:40	31.18
6:10-6:30	21.32
6:40-7:10	68.39
8:10-9:30	486.05
9:40-12:10	1150.75
12:40-14:40	461.35
15:30-17:00	198.89
17:30-17:50	32.16
18:10-19:20	239.07
20:00-20:50	106.46
21:10-22:10	161.52
23:00-23:10	8.29
23:40-0:10+1	63.06
2025-06-21 紧急购电时间段	购电量/kWh
0:30-0:40	44.34
1:20-2:20	142.23
2:30-2:40	9.36
3:00-3:40	110.46
4:10-4:30	64.53
4:50-5:30	69.31
6:10-6:20	14.90
6:30-7:30	93.37
13:10-13:20	18.54

2025-03-20 紧急购电时间段	购电量/kWh
17:40-18:00	29.32
19:00-19:20	57.43
19:50-20:00	0.60
20:30-20:40	29.16
21:20-21:50	112.68
22:20-22:40	26.43
23:20-23:30	1.30
2025-09-23 紧急购电时间段	购电量/kWh
0:40-1:20	68.77
1:40-1:50	7.87
2:10-2:30	52.90
3:30-3:40	5.66
5:00-5:30	74.13
5:50-7:30	194.83
7:50-8:10	76.65
8:30-9:00	125.50
9:30-10:30	144.49
10:50-11:10	58.63
11:40-12:10	87.67
13:00-15:10	331.68
16:00-16:50	185.54
17:10-17:30	15.62
17:40-18:00	15.02
18:10-18:50	88.20
19:00-19:10	12.45
19:30-19:40	2.71
20:00-20:40	150.89
21:00-21:50	166.43
22:00-22:10	1.05
22:30-22:40	34.92
23:40-23:50	12.31
2025-12-21 紧急购电时间段	购电量/kWh
0:10-0:40	38.86
1:10-2:00	106.07
2:10-2:30	40.84
3:10-3:30	86.11
3:40-4:30	201.58
5:10-5:40	47.20
5:50-6:10	65.84
6:20-6:40	53.68
7:10-7:40	49.56
7:50-8:20	69.97

2025-03-20 紧急购电时间段	购电量/kWh
8:30-8:40	17.65
9:00-9:10	12.74
9:40-10:10	92.39
10:40-12:20	818.26
12:50-13:20	35.37
14:00-14:10	23.53
16:30-16:40	7.02
17:00-17:10	11.98
17:30-17:40	7.18
18:10-19:10	208.44
19:30-20:10	80.18
20:40-21:00	24.26
21:20-21:30	1.08
21:50-22:10	6.73
22:50-23:20	51.71
23:40-0:00+1	56.33

表 8-4 说明滚动更新能够削减但不能完全消除紧急购电。剩余缺口主要出现在最后一次可用预报之后或实际净负荷突变较大的时段，符合“信息改善而非获得完全信息”的题意。

8.4 结果分析与检验

- **更新价值的判定准则：**新预报是否值得使用，必须由应急损失减少量与调整结算成本共同决定。四次更新使总费用减少 659704.73 元、紧急购电量下降 28.03%，表明在本组数据中信息收益超过调整成本；但新增 18:00 更新的边际节省已明显缩小，说明更新频率继续提高未必经济。
- **逐版账本的可复核性：**每次更新只针对尚未执行的时段，将新计划与上一版计划逐时段比较购电量，增加部分记为增购、减少部分记为减购；已执行时段保持不变，最终费用由计划购电、调整净费用和紧急购电共同构成。
- **约束与因果边界的遵守：**所有滚动节点均从当时实测 SOC 出发，状态递推、容量功率边界和充放电互斥均满足要求，且决策只使用发布时刻之前的信息。因此该策略既利用了日内新信息，又保持了真实可执行性。

九、问题四的模型建立与求解

9.1 价格数据处理与因果预测

附件 4 记录的是全年最终实现的实时电价，并不意味着每天 0:00 已经知道当天未来 144 个时段的价格。若直接使用当天未来真实价格制定购电和储能计划，将产生未来信息泄漏。因此，本文在每个决策时点设置历史数据截断，只使用该时点以前已经观测到的价格信息。

记日期附件 4 记录的是全年最终实现的实时电价，并不意味着每天 0:00 已经知道当天未来 144 个时段的价格。若直接使用当天未来真实价格制定购电和储能计划，将产生未来信息泄漏。因此，本文在每个决策时点设置历史数据截断，只使用该时点以前已经观测到的价格信息。记日期 d 、决策时刻 T 的可用价格信息集为：

$$\mathcal{P}_{d,T}^p = \{p_{i,t}: i < d\} \cup \{p_{d,t}: t \leq T\} \quad (17)$$

其中， $P_{d,t}$ 表示实际实时电价。未来时段的预测价格记为

$$\hat{p}_{d,t|T} = f(\mathcal{P}_{d,T}^p), \quad t > T \quad (18)$$

本文比较 1-7 日季节加权预测和历史同星期中位数预测，并根据决策日前的滚动验证误差选择预测方法。对于日内更新，还利用当日已经实现的价格前缀校正后续预测。模型训练、参数选择和误差计算均不使用决策时点以后的数据。

问题 4-2 在每天 0:00 预测当天全部实时价格，并据此求解问题二的日前两阶段模型；问题 4-3 则在 0:00、6:00、12:00 和 18:00 同步更新价格与光伏预测，并只调整尚未执行的计划。

9.2 调度模型与评价基准

问题四沿用前述母线能量平衡、储能状态转移、容量边界、功率限制和充放电互斥约束，只将原模型中的已知电价替换为决策时能够获得的预测价格。

对于问题 4-2，模型在每天 0:00 依据预测价格、净负荷预测和历史残差场景确定全天计划。实际负荷、光伏和价格实现后，最终费用按真实实时价格计算：

$$C_d^{4-2} = \sum_{t=1}^{144} p_{d,t} G_{d,t}^{(0)} + \sum_{t=1}^{144} 5 p_{d,t} R_{d,t} \quad (19)$$

其中， $G_{d,t}^{(0)}$ 为 0:00 确定的正常购电量， $R_{d,t}$ 为紧急购电量。预测价格仅用于制定计划，最终费用始终使用实际价格计算。

对于问题 4-3，在四个预测发布时点重新预测未来价格和光伏发电功率，并以当前 SOC 为初值滚动优化未执行时段。记第 h 次调整的增购量和减购量分别为 h

$$\Delta G_{d,t}^{+,h} = \max\{G_{d,t}^{(h)} - \bar{G}_{d,t}^{(h-)}, 0\} \quad (20)$$

$$\Delta G_{d,t}^{-,h} = \max\{\bar{G}_{d,t}^{(h-)} - G_{d,t}^{(h)}, 0\} \quad (21)$$

$$C_d^{4-3} = \sum_t p_{d,t} G_{d,t}^{(0)} + \sum_{h \in \{6,12,18\}} \sum_{t \geq h} (1.5 p_{d,t} \Delta G_{d,t}^{+,h} - 0.5 p_{d,t} \Delta G_{d,t}^{-,h}) + \sum_t 5 p_{d,t} R_{d,t} \bar{G}_{d,t}^{(h-)} \quad (22)$$

其中，增购电量按实际价格的 1.5 倍支付，减购电量退回实际价格的 50%，最终仍存在的供电缺口按实际价格的 5 倍紧急购电。

为评价未来价格信息的价值，本文设置以下两个对照基准。

（1）完全价格信息基准：只将未来真实价格提供给优化模型，负荷、光伏预测和计划调整规则保持不变。该基准用于度量价格预测误差造成的成本损失。

（2）全外生信息下界：同时提供真实价格、真实负荷和真实光伏路径，并放宽充放电互斥整数约束。该结果仅作为理论成本下界，不属于现实可执行方案。

9.3 模型求解与计算结果

在因果价格预测条件下，问题 4-2 和问题 4-3 的全年总费用分别为 16124373.46 元和 16550967.30 元。对应的完全价格信息基准费用分别为 15923572.41 元和 16401924.52 元，全外生信息下界为 12780453.61 元，结果见表 9-1。

表 9-1 波动电价下三类策略的费用对照			
策略	Q4-2 总费用/元	Q4-3 总费用/元	角色定位
因果在线策略	16124373.46	16550967.30	正式结果
完全价格信息基准	15923572.41	16401924.52	价格信息价值评价
全外生信息下界	12780453.36	12780453.36	严格成本下界

表 9-1 中，问题 4-2 和问题 4-3 的因果总费用分别比完全价格信息基准高 200801.05 元和 149042.78 元，对应 1.261%和 0.909%。全外生信息下界为 12780453.36 元，明显更低，但它同时使用现实决策时不可获得的未来价格、负荷和光伏，仅能作为理论边界。

由于价格信息损失相对全年总费用较小，单看表格不易辨认其量级。图 9-1 先展示三类信息条件下的成本层级，再单独放大因价格未知造成的差额。

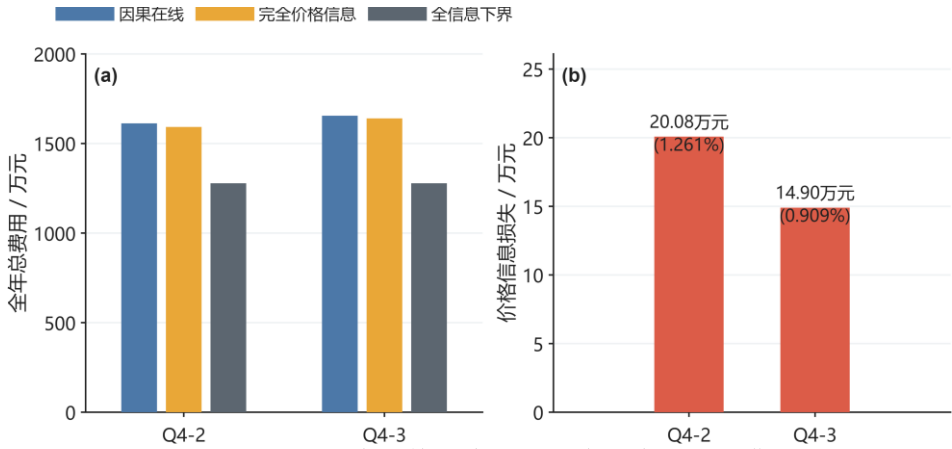


图 9-1 因果策略、完全价格信息基准与全信息下界的费用对比

图 9-1(a)显示两种因果策略均高于完全价格信息基准和全信息下界；图 9-1(b)进一步表明价格预测损失只占完全价格信息费用约 1%。这意味着储能和滚动计划已经缓冲了大部分价格不确定性，剩余较大的成本差距主要来自负荷、光伏等其他未来信息不可知。

年度汇总之外，表 9-2 列出四个指定日期在问题 4-2 与问题 4-3 下的总费用和紧急购电量，以检查结论是否由个别日期异常主导。

表 9-2 波动电价下指定日期结果				
日期	Q4-2 总费用/元	Q4-2 紧急购电 /kWh	Q4-3 总费用/元	Q4-3 紧急购电 /kWh
2025-03-20	49580.79	336.20	57894.95	3395.27
2025-06-21	20454.56	196.52	19433.16	823.95
2025-09-23	50611.49	1019.05	51825.88	1913.89
2025-12-21	78821.59	1037.31	80925.21	2214.58

表 9-2 显示，滚动价格与光伏更新在不同日期上的收益方向并不一致：部分日期降低费用，另一些日期因调整结算和预测偏差反而增加费用。由此可知，问题 4-3 全年费用高于问题 4-2 并非模型失效，而是题定非对称调整成本在波动电价下累积后的结果。

9.4 结果分析与信息价值

问题 4-2 相对于完全价格信息基准增加 200801.05 元，占基准费用的 1.26%；问题 4-3 增加 149042.78 元，占基准费用的 0.91%。这表明仅使用历史价格进行预测会产生一定额外成本，但预测模型已经捕捉了实时价格的主要周期变化，价格未知造成的损失总体约为 1%。

问题 4-3 的全年费用比问题 4-2 高 426593.84 元。其原因并非日内新信息没有作用，而是问题 4-3 需要对购电计划进行多次调整。新预测能够减少部分供电偏差和紧急购电，但增购溢价与减购损失也会增加交易成本。当预测更新带来的紧急购电节省不足以抵消计划调整成本时，滚动策略的总费用可能高于日前策略。

因此，在实时价格环境下，计划是否需要更新应由预测更新收益与计划调整成本的比较结果决定，而不能仅以是否获得新预测作为调整依据。本题数据下，问题 4-2 具有更低的全年费用，问题 4-3 则具有更强的信息响应能力。全外生信息下界与现实策略仍存在较大差距，说明剩余成本差异主要来源于负荷、光伏等未来信息不可知，而不仅是价格预测误差。

十、模型检验与结果可靠性

10.1 硬约束与费用账本复核

为检验计算结果的可靠性，本文编写独立复核程序，对四个问题输出的能量流、储能状态及费用结果进行重新计算，并逐项检查模型约束。复核项目及结果见表 10-1。

表 10-1 各问约束复核结果

检验项	问题一	问题二	问题三	问题四
母线功率平衡最大残差/kWh	2.27×10^{-13}	$\leq1\times10^{-6}$	8.20×10^{-12}	8.19×10^{-12}
储能状态转移最大残差/kWh	8.31×10^{-13}	$\leq1\times10^{-6}$	7.28×10^{-12}	1.25×10^{-10}
储电量是否落在 [1200, 10800]	是	是	是	是
同时充放电时段数	0	0	0	0
跨日储电量传递最大残差/kWh	0（首末相等）	$\leq1\times10^{-6}$	$\leq1\times10^{-6}$	$\leq1\times10^{-6}$
费用账本恒等式	通过	通过	通过	通过

由表 10-1 可知，问题一的母线电量平衡和储能状态转移残差分别为 2.27×10^{-13} kWh 和 8.31×10^{-13} kWh；问题三和问题四的相应残差均小于 1.3×10^{-10} kWh。问题二表中给出的残差均满足 1×10^{-6} kWh 的核验阈值。四个问题的储电量均处于 1200~10800 kWh 范围内，且不存在同时充电和放电的时段。

跨日状态复核表明，问题二至问题四中每日 24:00 的储电量均能够传递至下一日 0:00，最大传递残差不超过 1×10^{-6} kWh；问题一按照题目要求设置首末储电量相等。

费用账本方面，问题二至问题四逐日核对计划购电费、调整费用、紧急购电费与总费用之间的关系，问题一核对日购电费与总费用的一致性，所有结果均通过复核。

10.2 未来信息泄漏检验

含预测的调度模型必须保证决策只使用相应决策时点能够获得的信息。为此，本文从数据截断、参数估计和实际值使用三个方面进行检验。

首先，所有预测函数均设置历史数据截止索引，目标日的训练样本不得包含决策时刻之后的数据。其次，标准化参数、历史分位数、相似日距离、残差场景库及模型超参数均仅利用截止时刻以前的数据估计，并在每次预测时记录训练截止日期，检查训练样本是否早于预测原点。最后，附件 2 和附件 4 中尚未揭示的实际负荷、光伏发电功率及电价，只能在相应时段进入运行结算，不能用于此前的日前计划或滚动调整。

完全价格信息基准和全信息下界采用独立的计算函数及结果标识，仅用于反事实比较，不参与正式策略的预测和决策。因此，正式结果不存在将未来实际数据提前用于模型训练或调度的情况。

10.3 充放电互斥约束的必要性

问题一中，连续松弛模型与含整数变量的模型取得了相同目标值，但这并不意味着充放电互斥约束可以删除。连续松弛只说明在问题一给定数据下，最优解恰好没有出现同时充电和放电现象，不能保证这一性质在其他数据和运行场景下仍然成立。

设某一 10 min 时段的光伏发电量超过负荷用电量与单时段最大充电量之和。若允许储能同时充电和放电，模型可能令充电量为 C_t 、放电量为 D_t ，其中 D_t 由储能向交流母线供给， C_t 用于吸收光伏发电量。通过同时增大 C_t 和 D_t ，可以在基本不改变母线供需关系的情况下，使储能内部电量按照 $0.9C_t - \frac{D_t}{0.9}$ 减少。这样，储能损耗会被模型等价地用于消纳富余光伏电量，形成不符合实际运行方式的虚假消纳，并可能低估弃光量、购电量及购电费用。

因此，充放电互斥约束是保证模型物理可执行性的必要条件。即使本题数据下连续松弛与整数模型的最优费用相同，正式模型仍应保留二元状态变量，以排除其他数据条件下可能出现的非物理充放电行为。

10.4 情景数量与预测器选择的稳健性

问题二的情景数根据决策日前可获得的历史残差样本，在 10、20 和 30 个候选值中逐日确定。正式统计期共 334 个决策日，其中 331 日采用 30 个情景，2 日采用 10 个情景，1 日采用 20 个情景。

上述情景数分布可以由逐日选择日志直接复核，说明模型能够根据可用历史样本量调整情景规模，在保持整日残差轨迹代表性的同时，避免对有限历史样本进行无依据扩充。由于未形成独立、完整的固定情景数对照计算记录，本文不进一步比较不同固定情景数下的费用差异。

预测器选择采用简单优先规则：当候选模型的 WAPE 差异不超过 1%时，选择结构较简单的模型。正式统计期净负荷预测的 WAPE 为 10.95%，多数月份选择季节性加

权基线模型 N1。该结果表明，在本数据集和当前验证方法下，结构更复杂的模型未表现出稳定的预测优势，简单优先规则能够在预测效果与模型复杂度之间取得合理平衡。

十一、模型评价、改进与推广

11.1 模型优点

(1) 模型结构与题意高度一致。本文从微电网逐时段电量平衡和储能状态转移关系出发，根据四个问题中信息条件、决策时点与费用结算规则的变化逐步扩展模型。四问采用相同的物理约束体系，同时分别刻画确定性计划、日前不确定性、日内滚动调整和波动电价，模型之间衔接自然，避免了各问相互割裂。

(2) 严格遵守决策的信息边界。问题二将 0:00 制定的计划与实际运行后的紧急购电明确分离；问题三在每次预报更新时固定已执行决策和当前储电量，仅优化尚未执行的时段；问题四在本文设定的实时价格信息条件下，仅利用决策时点以前的价格数据预测未来价格。由此保证各项策略均能按照真实时间顺序实施，避免使用事后信息造成结果虚优。

(3) 模型可解释性与求解可靠性较强。各项决策变量均具有明确的物理或经济含义，目标函数可以分解为计划购电、调整购电和紧急购电等费用。各问最终均转化为混合整数线性规划问题，可由成熟求解器稳定求解；在给定模型、数据和求解精度下，分支定界法能够提供具有最优性保证的解，而不依赖随机初始化或启发式搜索。

(4) 物理约束与计算结果能够相互验证。本文对母线电量平衡、储能状态递推、容量边界、充放电功率限制和充放电互斥关系进行了逐时段回代，并连续传递相邻日期之间的储电量。计算结果满足相应约束，论文中的主要费用、购电量和储能状态均可由程序输出与结果文件复算。

(5) 对照基准具有明确的解释作用。本文不仅报告现实信息条件下的正式策略，还设置无储能方案、完全价格信息基准和全外生信息下界等对照，分别衡量储能调节价值、价格预测损失以及未来信息不可知造成的成本差距，从而使模型效果的来源能够被区分和解释。

11.2 模型缺点

(1) 历史样本对极端风险的覆盖有限。附件数据仅包含一个年度，虽然能够反映日内周期和一定的季节变化，但不足以充分描述跨年度气候漂移、连续极端天气以及异常价格尖峰。因此，依据历史残差构造的预测和场景可能低估小概率、高损失事件的风险。

(2) 时间离散会忽略更短周期的运行变化。本文以 10 分钟为一个决策时段，并将时段内的负荷、光伏出力及充放电功率视为稳定值，未进一步描述秒级或分钟级功率波动、储能爬坡过程与控制延迟。在更高实时性要求下，该简化可能低估瞬时功率失衡。

(3) 经济目标尚未覆盖全部设备成本。题目没有提供电池循环衰减、启停损耗、最大需量电费和上网交易参数，因此本文主要优化购电与偏差结算费用。当储能寿命损耗或容量费用不可忽略时，现有目标函数对长期运行经济性的刻画仍不完整。

11.3 模型改进

针对极端样本覆盖不足的问题，可在获得多年气象和运行数据后，按照季节、天气类型和预测提前量分别估计条件残差分布，并采用滚动回测选择场景数量及尾部样本保留比例。若引入极端场景后能够显著降低样本外高额费用，可进一步在目标函数中加入条件风险价值，或采用机会约束限制供电不足概率。

针对时间离散带来的误差，可构建“日前 10 分钟调度—日内分钟级修正”的分层控制框架：上层模型负责购电和储能能量安排，下层模型根据实时功率偏差修正储能指令。这样既能保留全年优化的计算效率，又能提高策略对短时波动和控制延迟的适应能力。

针对设备成本缺失的问题，可依据电池厂商提供的循环寿命曲线建立分段线性衰减费用，并将启停损耗、最大需量费用及允许上网时的售电收益纳入统一费用账本。新增参数应先进行可辨识性和灵敏度检验，避免以缺乏依据的主观权重替代真实工程数据。

11.4 模型推广

本文形成的“物理约束—信息结构—结算机制”建模思路具有较好的可迁移性。对于含风电的园区微网、电动汽车充电站、建筑综合能源系统和数据中心备用储能等问题，可先建立能量平衡与状态转移关系，再根据信息的发布时间确定哪些决策必须提前固定、哪些决策能够随新信息更新，最后按照实际交易和偏差结算规则构造目标函数。

模型推广的前提是系统状态能够被连续观测，设备约束和交易规则可以明确量化，并且预测数据与实际执行时刻严格对应。应用于不同场景时，需要相应替换能源转换关系、设备运行边界和费用参数，而不宜直接照搬本文参数。

十二、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于赛题理解辅助、候选模型比较、代码编写与调试、结果核验、图表生成和文字润色，详细使用情况见支撑材料。

十三、参考文献

- [1] 张亚琳. 具有电池储能系统的微电网分布式经济调度方案与 SoC 均衡控制[D]. 天津: 南开大学, 2025. DOI:10.27254/d.cnki.gnka.2025.000023.
- [2] 李嘉伟, 巨云涛, 张璐, 等. 基于分布鲁棒模型预测控制的微电网多时间尺度优化调度[J]. 电力工程技术, 2024, 43(4):45-55. DOI:10.12158/j.2096-3203.2024.04.005.
- [3] 张理, 王宝, 贾健雄, 等. 微电网功率预测与调度端到端协同优化方法[J]. 上海交通大学学报, 2025, 59(6):720-731. DOI:10.16183/j.cnki.jsjtu.2024.224.

[4] LIU C, QIN Y, ZHANG H. Real-time scheduling strategy for microgrids considering operation intervaldivision of DGs and batteries[J]. Global Energy Interconnection, 2020, 3(5):442-452. DOI:10.1016/j.gloi.2020.11.004.

[5] OROZCO C, BORGHETTI A, DE SCHUTTER B, et al. Intra-day scheduling of a local energy community coordinated with day-ahead multistage decisions[J]. Sustainable Energy, Grids and Networks, 2022, 29:100573. DOI:10.1016/j.segan.2021.100573.

[6] ELKAZAZ M, SUMNER M, THOMAS D. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J]. International Journal of Electrical Power & Energy Systems, 2020, 115:105483. DOI:10.1016/j.ijepes.2019.105483.

[7] 侯慧, 王晴, 薛梦雅, 等. 计及源荷不确定性及需求响应的离网型微电网两阶段日前经济调度[J]. 电力系统保护与控制, 2022, 50(13):73-85. DOI:10.19783/j.cnki.pspc.211406.

[8] 刘一欣, 郭力, 王成山. 微电网两阶段鲁棒优化经济调度方法[J]. 中国电机工程学报, 2018, 38(14):4013-4022, 4307. DOI:10.13334/j.0258-8013.pcsee.170500.

[9] 王灿, 张雪菲, 凌凯, 等. 基于区间概率不确定集的微电网两阶段自适应鲁棒优化调度[J]. 中国电机工程学报, 2024, 44(5):1750-1764. DOI:10.13334/j.0258-8013.pcsee.221968.

十四、附录

附录 A 给出支撑材料清单与运行说明，附录 B 给出与四问结果直接对应的完整求解链源程序。

附录 A 支撑材料的文件列表

支撑材料按“主程序—必要公共模块—核验与导出—结果与图件—使用说明”的顺序组织，不包含赛题原始附件、论文旧稿、排版脚本或本机路径。文件清单如下。

文件名	功能描述
code/q1_model.py	问题一确定性调度、连续松弛与无储能对照
code/q2_model.py	问题二严格日前两阶段随机调度
code/q3_model.py	问题三光伏预报更新下的滚动调度
code/q4_model.py	问题四因果实时价格策略与信息基准
code/common_milp.py	四问共用的能量平衡与储能调度求解器
code/data_loader.py	问题一数据读取与单位转换
code/formal_data_loader.py	全年数据读取、字段校验与单位转换
code/causal_forecasts.py	因果预测与历史残差情景生成
code/q34_helpers.py	问题三、四的滚动状态更新与费用结算
code/verify_results.py	四问能量平衡、状态转移、互斥与费用复核
code/export_result_workbooks.py	计算结果工作簿导出
code/export_matlab_visual_data.py	MATLAB 绘图数据导出
code/run_all.py	统一核验与完整复算入口
code/plot_q1_dispatch_matlab_preview.m	问题一调度与储能状态图
code/plot_c_paper_figures_matlab.m	问题二至问题四证据图
代码运行说明.txt	运行环境与复算步骤
requirements.txt	Python 依赖清单

文件名	功能描述
AI 工具使用详情.pdf	AI 工具使用与人工核验记录
results/	四问结果与核验数据
results/figures_matlab/	MATLAB 图件与可编辑 FIG 文件

运行方式：按《代码运行说明》放置赛题附件后，在 code 目录执行 python run_all.py 进行求解、核验与导出，执行 python run_all.py --full 进行全年完整复算，执行 python run_all.py --check 仅核验已有结果。MATLAB 图件由 plot_q1_dispatch_matlab_preview.m 与 plot_c_paper_figures_matlab.m 生成。

附录 B 源程序代码

q1_model.py 问题一：确定性 MILP 主模型与 LP 连续松弛对照

```
1  """问题一正式求解：确定性 MILP 主模型与 LP 连续松弛对照。"""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11 from common_milp import solve_deterministic_dispatch
12 from data_loader import ROOT, load_q1
13
35 def main() -> None:
36     data = load_q1()
37     kwargs = dict(
38         load_kwh=data["load_kwh"], pv_kwh=data["pv_kwh"],
39         price=data["price"],
40         initial_soc=6000.0, terminal_soc=6000.0,
41         allow_emergency=False,
42     )
43     milp_result = solve_deterministic_dispatch(**kwargs,
44         binary_mode=True)
45     lp_result = solve_deterministic_dispatch(**kwargs,
46         binary_mode=False)
```

```

44     selected_labels = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
45 "16:00-16:10", "18:00-18:10", "20:00-20:10"]
46     # 同序号对齐下, 10:00-10:10 对应附件第 60 项 (数组下标 59) , 其后每 2
47     小时增加 12 项。
48     selected_indices = [59, 71, 83, 95, 107, 119]
49     selected = {
50         label: float(milp_result.grid[idx])
51         for label, idx in zip(selected_labels, selected_indices)
52     }
53     blocks = []
54     for start in range(0, 144, 24):
55         blocks.append({
56             "interval": f"{start // 6}:00-{(start + 24) // 6}:00",
57             "charge_kwh": float(milp_result.charge[start:start +
58 24].sum()),
59             "discharge_kwh": float(milp_result.discharge[start:start
60 + 24].sum()),
61             "soc_end_kwh": float(milp_result.soc[start + 24]),
62         })
63
64     no_storage_cost = float(np.dot(data["price"],
65 np.maximum(data["load_kwh"] - data["pv_kwh"], 0.0)))
66
67     report = {
68         "question": "Q1",
69         "model": "deterministic_MILP_with_charge_discharge_mutex",
70         "status": "success",
71         "source_files": ["problems/选题 C_2026 正式/附件/附件 1.xlsx"],
72         "time_step_hours": 1.0 / 6.0,
73         "alignment_rule": "附件与结果模板按同序号对应, 不循环平移",
74         "milp": _summary(milp_result),
75         "lp_relaxation": _summary(lp_result),
76         "lp_lower_bound_gap_yuan": float(milp_result.objective -
77 lp_result.objective),
78         "no_storage_baseline_cost_yuan": no_storage_cost,
79         "cost_saving_vs_no_storage_yuan": float(no_storage_cost -
80 milp_result.objective),

```

```

73     "cost_saving_rate": float((no_storage_cost -
milp_result.objective) / no_storage_cost),
74     "selected_interval_grid_kwh": selected,
75     "four_hour_blocks": blocks,
76     "validation": {
77         "balance_residual_le_1e-6":
bool(milp_result.max_balance_residual <= 1e-6),
78         "soc_residual_le_1e-6":
bool(milp_result.max_soc_residual <= 1e-6),
79         "soc_bounds": bool(milp_result.soc.min() >= 1200 - 1e-6
and milp_result.soc.max() <= 10800 + 1e-6),
80         "terminal_soc": bool(abs(milp_result.soc[-1] - 6000) <=
1e-6),
81         "no_simultaneous": bool(np.all((milp_result.charge <=
1e-7) | (milp_result.discharge <= 1e-7))),
82         "lp_lower_bound": bool(lp_result.objective <=
milp_result.objective + 1e-6),
83     },
84     "claims": [
85         {
86             "claim_id": "Q1-COST",
87             "value": float(milp_result.objective),
88             "unit": "元/日",
89             "code": "code/q1_model.py",
90             "data": "problems/选题 C_2026 正式/附件/附件 1.xlsx",
91         }
92     ],
93 }
94 report["status"] = "success" if
all(report["validation"].values()) else "failed"

```

q2_model.py 问题二：严格 0 时两阶段随机 MILP

- 1 """问题二：严格 0 时两阶段随机 MILP 全年滚动求解。
- 2
- 3 第一阶段：计划购电 G、充电 C、放电 D、SOC E 与互斥状态 z 跨情景共享。
- 4 第二阶段：每个净负荷情景只允许紧急购电 R 和弃电 W。

```
5 输入：附件 1 固定电价与附件 2 全年实际负荷/光伏。
6 输出：state/agent_outputs/q2_results.json、results/q2_detail.npz。
7 运行：python code/q2_model.py [--days N] [--horizon-days 1|2]
8 """
9
10 from __future__ import annotations
11
12 import argparse
13 import hashlib
14 import json
15 import platform
16 from dataclasses import asdict, dataclass
17 from datetime import datetime, timezone
18 from pathlib import Path
19 from time import perf_counter
20
21 import numpy as np
22 import scipy
23 from scipy.optimize import Bounds, LinearConstraint, milp
24 from scipy.sparse import lil_matrix
25
26 from causal_forecasts import (
27     ForecastChoice,
28     choose_scenario_count,
29     forecast_from_choice,
30     residual_scenarios,
31     select_month_model,
32 )
33 from formal_data_loader import DATA_DIR, N_STEPS, load_q2_data,
34     load_typical_day
35
36 ROOT = Path(__file__).resolve().parents[1]
37 RESULTS_DIR = ROOT / "results"
38 STATE_DIR = ROOT / "state" / "agent_outputs"
39 ETA_C = 0.9
```

```

40 ETA_D = 0.9
41 SOC_MIN = 1200.0
42 SOC_MAX = 10800.0
43 POWER_LIMIT_KWH = 5000.0 / 6.0
44 INITIAL_SOC = 6000.0
45 EMERGENCY_MULTIPLIER = 5.0
46 # .....
47
48 @dataclass
49 class StochasticDispatch:
50     grid: np.ndarray
51     charge: np.ndarray
52     discharge: np.ndarray
53     soc: np.ndarray
54     mode: np.ndarray
55     scenario_emergency: np.ndarray
56     scenario_curtailment: np.ndarray
57     objective: float
58     solve_seconds: float
59     success: bool
60     message: str
61     mip_gap: float | None
62     max_balance_residual: float
63     max_soc_residual: float
64
65
66 def solve_two_stage_day(
67     scenario_net_kwh: np.ndarray,
68     probabilities: np.ndarray,
69     price: np.ndarray,
70     initial_soc: float,
71     *,
72     terminal_soc: float | None = None,
73     time_limit: float = 60.0,
74 ) -> StochasticDispatch:
75     """求解共享计划、仅 R/W 追索的两阶段随机 MILP。"""
76

```

```

77     scenarios = np.asarray(scenario_net_kwh, dtype=float)
78     probs = np.asarray(probabilities, dtype=float)
79     tariff = np.asarray(price, dtype=float)
80     if scenarios.ndim != 2:
81         raise ValueError("scenario_net_kwh 必须为 S×H 矩阵")
82     s_count, horizon = scenarios.shape
83     if tariff.shape != (horizon,) or probs.shape != (s_count,):
84         raise ValueError("电价、概率与情景维度不匹配")
85     if np.any(probs < 0) or not np.isclose(probs.sum(), 1.0):
86         raise ValueError("情景概率必须非负且和为 1")
87     if np.any(tariff < 0) or not np.isfinite(scenarios).all():
88         raise ValueError("电价不得为负, 净负荷情景必须有限")
89
90     # g,c,d,E,z,R,W
91     g0 = 0
92     c0 = g0 + horizon
93     d0 = c0 + horizon
94     e0 = d0 + horizon
95     z0 = e0 + horizon + 1
96     r0 = z0 + horizon
97     w0 = r0 + s_count * horizon
98     nvar = w0 + s_count * horizon
99
100    objective = np.zeros(nvar)
101    objective[g0 : g0 + horizon] = tariff
102    for s in range(s_count):
103        lo = r0 + s * horizon
104        objective[lo : lo + horizon] = probs[s] *
EMERGENCY_MULTIPLIER * tariff
105
106    lower = np.zeros(nvar)
107    upper = np.full(nvar, np.inf)
108    upper[c0 : c0 + horizon] = POWER_LIMIT_KWH
109    upper[d0 : d0 + horizon] = POWER_LIMIT_KWH
110    lower[e0 : e0 + horizon + 1] = SOC_MIN
111    upper[e0 : e0 + horizon + 1] = SOC_MAX

```

```

112     lower[e0] = upper[e0] = float(initial_soc)
113     if terminal_soc is not None:
114         lower[e0 + horizon] = upper[e0 + horizon] =
float(terminal_soc)
115     upper[z0 : z0 + horizon] = 1.0
116
117     # 每个情景每时段平衡 + 共享 SOC 状态转移。
118     aeq = lil_matrix((s_count * horizon + horizon, nvar),
dtype=float)
119     beq = np.zeros(s_count * horizon + horizon)
120     for s in range(s_count):
121         for t in range(horizon):
122             row = s * horizon + t
123             aeq[row, g0 + t] = 1.0
124             aeq[row, c0 + t] = -1.0
125             aeq[row, d0 + t] = 1.0
126             aeq[row, r0 + s * horizon + t] = 1.0
127             aeq[row, w0 + s * horizon + t] = -1.0
128             beq[row] = scenarios[s, t]
129     for t in range(horizon):
130         row = s_count * horizon + t
131         aeq[row, e0 + t] = -1.0
132         aeq[row, e0 + t + 1] = 1.0
133         aeq[row, c0 + t] = -ETA_C
134         aeq[row, d0 + t] = 1.0 / ETA_D
135
136     # 充放电互斥:  $C \leq Mz$ ,  $D \leq M(1-z)$ 。
137     aub = lil_matrix((2 * horizon, nvar), dtype=float)
138     bub = np.zeros(2 * horizon)
139     for t in range(horizon):
140         aub[t, c0 + t] = 1.0
141         aub[t, z0 + t] = -POWER_LIMIT_KWH
142         aub[horizon + t, d0 + t] = 1.0
143         aub[horizon + t, z0 + t] = POWER_LIMIT_KWH
144         bub[horizon + t] = POWER_LIMIT_KWH
145

```

```

146     integrality = np.zeros(nvar, dtype=int)
147     integrality[z0 : z0 + horizon] = 1
148     started = perf_counter()
149     raw = milp(
150         objective,
151         integrality=integrality,
152         bounds=Bounds(lower, upper),
153         constraints=[
154             LinearConstraint(aeq.tocsr(), beq, beq),
155             LinearConstraint(aub.tocsr(), -np.inf, bub),
156         ],
157         options={"time_limit": time_limit, "mip_rel_gap": 1e-7},
158     )
159     elapsed = perf_counter() - started
160     if raw.x is None:
161         raise RuntimeError(f"Q2 两阶段 MILP 无可解: {raw.message}")
162     x = raw.x
163     grid = x[g0 : g0 + horizon]
164     charge = x[c0 : c0 + horizon]
165     discharge = x[d0 : d0 + horizon]
166     soc = x[e0 : e0 + horizon + 1]
167     mode = x[z0 : z0 + horizon]
168     emergency = x[r0 : r0 + s_count * horizon].reshape(s_count,
169     horizon)
170     curtailment = x[w0 : w0 + s_count * horizon].reshape(s_count,
171     horizon)
172     balance = (
173         grid[None, :]
174         - charge[None, :]
175         + discharge[None, :]
176         + emergency
177         - curtailment
178         - scenarios
179     )
180     soc_residual = soc[1:] - soc[:-1] - ETA_C * charge + discharge /
181     ETA_D

```

```

179     return StochasticDispatch(
180         grid=grid,
181         charge=charge,
182         discharge=discharge,
183         soc=soc,
184         mode=mode,
185         scenario_emergency=emergency,
186         scenario_curtailment=curtailment,
187         objective=float(raw.fun),
188         solve_seconds=elapsed,
189         success=bool(raw.success),
190         message=str(raw.message),
191         mip_gap=float(raw.mip_gap) if getattr(raw, "mip_gap", None)
192         is not None else None,
193         max_balance_residual=float(np.max(np.abs(balance))),
194         max_soc_residual=float(np.max(np.abs(soc_residual))),
195     )

```

q3_model.py 问题三：0/6/12/18 滚动更新的确定性 MILP-MPC

```

1  """问题三：0/6/12/18 时四次光伏预测驱动的不确定性 MILP-MPC。
2
3  输入：附件 2 实际负荷/光伏、附件 3 分时发布光伏预测、附件 1 固定电价。
4  输出：results/q3_results.json、q3_daily.csv、q3_detail.npz。
5  运行：python code/q3_model.py --full --ablation
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 from datetime import datetime
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17

```

```

18 from q34_helpers import (
19     ORIGIN_SLOTS,
20     load_c_data,
21     select_load_params,
22     simulate_mpc,
23     summarize_run,
24 )
25
26
27 ROOT = Path(__file__).resolve().parents[1]
28 ATTACHMENTS = ROOT / "problems" / "选题 C_2026 正式" / "附件"
29 RESULTS = ROOT / "results"
30 STATE_OUTPUTS = ROOT / "state" / "agent_outputs"
31 SPECIFIED_DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-
    12-21")
    # .....
52 def run(full: bool, ablation: bool) -> dict:
53     RESULTS.mkdir(parents=True, exist_ok=True)
54     data = load_c_data(ATTACHMENTS)
55     nday = len(data.dates) if full else min(14, len(data.dates))
56     days = range(nday)
57     arrays, _, versions = simulate_mpc(
58         data,
59         days,
60         6000.0,
61         ORIGIN_SLOTS,
62         _price_provider(data),
63         np.tile(data.fixed_price, (len(data.dates), 1)),
64         keep_versions=True,
65     )
66     report_start = 31 if nday > 31 else 0
67     main_summary = summarize_run(arrays, report_start)
68
69     comparisons = {"0_6_12_18": main_summary}
70     if ablation:
71         for name, origins in {

```

```

72         "0_only": (0,),
73         "0_6": (0, 36),
74         "0_6_12": (0, 36, 72),
75     }.items():
76         comp, _, _ = simulate_mpc(
77             data,
78             days,
79             6000.0,
80             origins,
81             _price_provider(data),
82             np.tile(data.fixed_price, (len(data.dates), 1)),
83         )
84         comparisons[name] = summarize_run(comp, report_start)
85
86     dates = data.dates[:nday]
87     daily = pd.DataFrame(
88         {
89             "date": dates.strftime("%Y-%m-%d"),
90             "planned_cost_yuan": arrays["planned_cost"],
91             "adjustment_cost_yuan": arrays["adjustment_cost"],
92             "emergency_cost_yuan": arrays["emergency_cost"],
93             "total_cost_yuan": arrays["total_cost"],
94             "emergency_energy_kwh": arrays["emergency"].sum(axis=1),
95             "grid_energy_kwh": arrays["effective_grid"].sum(axis=1),
96             "minimum_soc_kwh": arrays["soc"].min(axis=1),
97             "solve_seconds": arrays["solve_seconds"],
98         }
99     )
100     daily.to_csv(RESULTS / "q3_daily.csv", index=False,
101                 encoding="utf-8-sig")
102     grid_versions, load_forecasts, pv_forecasts =
103     _version_npz(versions, nday)
104     np.savez_compressed(
105         RESULTS / "q3_detail.npz",
106         dates=dates.strftime("%Y-%m-%d").to_numpy(),
107         **arrays,

```

```

106         grid_versions=grid_versions,
107         load_forecasts=load_forecasts,
108         pv_forecasts=pv_forecasts,
109     )

```

q4_model.py 问题四：实时波动电价下的因果 Q4-2/Q4-3 与信息基准

```

1  """问题四：实时波动电价下的因果 Q4-2/Q4-3 与完全价格信息基准。
2
3  因果策略的预测、选参与调度均只访问决策时点以前的价格；真实价格仅用于
4  事后结算。输出：results/q4_results.json、q4_daily.csv、
5      q4_detail.npz。
6  运行：python code/q4_model.py --full
7  """
8  from __future__ import annotations
9
10 import argparse
11 import json
12 from datetime import datetime
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17
18 from q34_helpers import (
19     ORIGIN_SLOTS,
20     load_c_data,
21     predict_price,
22     select_price_params,
23     simulate_mpc,
24     summarize_run,
25 )
26 from q2_model import solve_two_stage_day
27 from causal_forecasts import (
28     ForecastChoice,
29     choose_scenario_count,

```

```
30     forecast_from_choice,
31     residual_scenarios,
32     select_month_model,
33 )
34
35
36 ROOT = Path(__file__).resolve().parents[1]
37 ATTACHMENTS = ROOT / "problems" / "选题 C_2026 正式" / "附件"
38 RESULTS = ROOT / "results"
39 STATE_OUTPUTS = ROOT / "state" / "agent_outputs"
40 SPECIFIED_DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-
    12-21")
41
42 # .....
43 class CausalPriceProvider:
44     def __init__(self, data):
45         self.data = data
46         self.cache = {}
47
48     def __call__(self, day: int, origin: int) -> np.ndarray:
49         key = (day, origin)
50         if key not in self.cache:
51             selected = select_price_params(self.data, day, origin)
52             self.cache[key] = (selected, predict_price(self.data,
53                 day, origin, selected))
54         return self.cache[key][1].copy()
55
56 class PerfectPriceProvider:
57     def __init__(self, data):
58         self.data = data
59
60     def __call__(self, day: int, origin: int) -> np.ndarray:
61         return self.data.realtime_price[day, origin:].copy()
62
63
```

```

64 def simulate_q42(data, nday: int, *, perfect_price: bool = False):
65     """Q4-2: 保持 Q2 严格两阶段边界, 仅替换为实时价格信息。"""
66     net = data.load - data.pv_actual
67     shape = (nday, 144)
68     out = {
69         "initial_grid": np.zeros(shape), "effective_grid":
70         np.zeros(shape),
71         "charge": np.zeros(shape), "discharge": np.zeros(shape),
72         "emergency": np.zeros(shape), "curtailment":
73         np.zeros(shape),
74         "soc": np.zeros((nday, 145)), "planned_cost":
75         np.zeros(nday),
76         "adjustment_cost": np.zeros(nday), "emergency_cost":
77         np.zeros(nday),
78         "total_cost": np.zeros(nday), "solve_seconds":
79         np.zeros(nday),
80         "max_balance_residual": np.zeros(nday), "max_soc_residual":
81         np.zeros(nday),
82     }
83     current_soc = 6000.0
84     choice = ForecastChoice("N1", {"alpha": 1.0}, float("nan"),
85                             float("nan"), 0)
86     residual_library = []
87     for day in range(nday):
88         if day >= 31 and data.dates[day].day == 1:
89             choice = select_month_model(net, data.load,
90                                         data.pv_actual, day, data.typical_load - data.typical_pv)
91             point1 = (
92                 data.typical_load - data.typical_pv if day == 0 else
93                 forecast_from_choice(choice, net, data.load,
94                                     data.pv_actual, day, day, data.typical_load - data.typical_pv)
95             )
96             residuals = np.vstack(residual_library) if residual_library
97             else np.empty((0, 144))
98             count = choose_scenario_count(residuals) if len(residuals)
99             else 1

```

```

89     scen1, probs, selected = residual_scenarios(point1,
residuals, scenario_count=count)
90     point2 = forecast_from_choice(
91         choice, net, data.load, data.pv_actual, day + 1, day,
92         data.typical_load - data.typical_pv,
lag1_override=point1,
93     )
94     scen2 = point2[None, :] + residuals[selected] if
len(residuals) else point2[None, :]
95     scenarios = np.hstack([scen1, scen2])
96     if perfect_price:
97         p1 = data.realtime_price[day]
98         p2 = data.realtime_price[day + 1] if day + 1 < nday else
p1
99     else:
100         selected_price = select_price_params(data, day, 0)
101         p1 = predict_price(data, day, 0, selected_price)
102         # 次日价格在当日 0 时仍不可见, 采用当前预测曲线延伸, 避免未来泄
漏。
103         p2 = p1.copy()
104         dispatch = solve_two_stage_day(
105             scenarios, probs, np.r_[p1, p2], current_soc,
terminal_soc=current_soc
106         )
107         g, c, d = dispatch.grid[:144], dispatch.charge[:144],
dispatch.discharge[:144]
108         soc = dispatch.soc[:145]
109         gap = net[day] + c - g - d
110         emergency = np.maximum(gap, 0.0)
111         curtailment = np.maximum(-gap, 0.0)
112         out["initial_grid"][day] = out["effective_grid"][day] = g
113         out["charge"][day], out["discharge"][day] = c, d
114         out["emergency"][day], out["curtailment"][day] = emergency,
curtailment
115         out["soc"][day] = soc
116         actual_price = data.realtime_price[day]

```

```

117         out["planned_cost"][day] = float(np.dot(actual_price, g))
118         out["emergency_cost"][day] = float(np.dot(5.0 *
actual_price, emergency))
119         out["total_cost"][day] = out["planned_cost"][day] +
out["emergency_cost"][day]
120         out["solve_seconds"][day] = dispatch.solve_seconds
121         out["max_balance_residual"][day] = max(
122             dispatch.max_balance_residual,
123             float(np.max(np.abs(g + data.pv_actual[day] + d +
emergency - data.load[day] - c - curtailment))),
124         )
125         out["max_soc_residual"][day] = dispatch.max_soc_residual
126         current_soc = float(soc[-1])
127         residual_library.append(net[day] - point1)
128     return out
129
130
131 def simulate_full_information_oracle(data, nday: int):
132     """实际负荷、光伏和价格均完全已知时的全年连续松弛下界。
133
134     一次性放宽全年信息边界，并进一步放松充放电二元变量；因此该目标值
135     不高于具有因果信息和整数互斥约束的现实策略。末端 SOC 不加人为约束。
136     """
137     shape = (nday, 144)
138     out = {
139         "initial_grid": np.zeros(shape), "effective_grid":
np.zeros(shape),
140         "charge": np.zeros(shape), "discharge": np.zeros(shape),
141         "emergency": np.zeros(shape), "curtailment":
np.zeros(shape),
142         "soc": np.zeros((nday, 145)), "planned_cost":
np.zeros(nday),
143         "adjustment_cost": np.zeros(nday), "emergency_cost":
np.zeros(nday),
144         "total_cost": np.zeros(nday), "solve_seconds":
np.zeros(nday),

```

```

145     "max_balance_residual": np.zeros(nday), "max_soc_residual":
np.zeros(nday),
146 }
147 solved = __import__("common_milp").solve_deterministic_dispatch(
148     data.load[:nday].reshape(-1),
149     data.pv_actual[:nday].reshape(-1),
150     data.realtime_price[:nday].reshape(-1),
151     6000.0, terminal_soc=None, allow_emergency=False,
binary_mode=False,
152     time_limit=600.0,
153 )
154 for day in range(nday):
155     lo, hi = 144 * day, 144 * (day + 1)
156     g, c, d = solved.grid[lo:hi], solved.charge[lo:hi],
solved.discharge[lo:hi]
157     soc = solved.soc[lo:hi + 1]
158     out["initial_grid"][day] = out["effective_grid"][day] = g
159     out["charge"][day], out["discharge"][day] = c, d
160     out["curtailment"][day] = solved.curtailment[lo:hi]
161     out["soc"][day] = soc
162     out["planned_cost"][day] =
float(np.dot(data.realtime_price[day], g))
163     out["total_cost"][day] = out["planned_cost"][day]
164     out["solve_seconds"][day] = solved.solve_seconds / max(nday,
1)
165     out["max_balance_residual"][day] = float(np.max(np.abs(
166         g + data.pv_actual[day] + d - data.load[day] - c -
out["curtailment"][day]
167     )))
168     out["max_soc_residual"][day] = float(np.max(np.abs(
169         soc[1:] - soc[:-1] - 0.9 * c + d / 0.9
170     )))
171     return out

```

common_milp.py 四问共用的能量平衡与储能调度 MILP 骨架

1 """微网调度的公共混合整数线性规划骨架。

```

2
3 变量统一使用每个离散时段内的电量 (kWh) 。储能充电量为交流母线侧
4 输入电量，放电量为交流母线侧输出电量，因此状态转移为
5  $E[t+1] = E[t] + \eta_c * \text{charge}[t] - \text{discharge}[t] / \eta_d.$ 
6 """
7
8 from __future__ import annotations
9
10 from dataclasses import dataclass
11 from time import perf_counter
12
13 import numpy as np
14 from scipy.optimize import Bounds, LinearConstraint, milp
15 from scipy.sparse import lil_matrix
16
17 # .....
18 @dataclass
19 class DispatchResult:
20     grid: np.ndarray
21     charge: np.ndarray
22     discharge: np.ndarray
23     curtailment: np.ndarray
24     emergency: np.ndarray
25     soc: np.ndarray
26     mode: np.ndarray
27     objective: float
28     solve_seconds: float
29     max_balance_residual: float
30     max_soc_residual: float
31     success: bool
32     message: str
33
34
35 def solve_deterministic_dispatch(
36     load_kwh: np.ndarray,
37     pv_kwh: np.ndarray,

```

```

38     price: np.ndarray,
39     initial_soc: float,
40     *,
41     terminal_soc: float | None = None,
42     eta_c: float = 0.9,
43     eta_d: float = 0.9,
44     soc_min: float = 1200.0,
45     soc_max: float = 10800.0,
46     max_power_kw: float = 5000.0,
47     step_hours: float = 1.0 / 6.0,
48     allow_emergency: bool = False,
49     emergency_multiplier: float = 5.0,
50     binary_mode: bool = True,
51     time_limit: float = 120.0,
52 ) -> DispatchResult:
53     """求解一个确定性时域内的购电与储能联合调度。"""
54
55     load = np.asarray(load_kwh, dtype=float)
56     pv = np.asarray(pv_kwh, dtype=float)
57     tariff = np.asarray(price, dtype=float)
58     if not (load.ndim == pv.ndim == tariff.ndim == 1):
59         raise ValueError("load_kwh、pv_kwh 和 price 必须是一维数组")
60     if not (len(load) == len(pv) == len(tariff)):
61         raise ValueError("负荷、光伏和电价长度必须一致")
62     if np.any(load < 0) or np.any(pv < 0) or np.any(tariff < 0):
63         raise ValueError("负荷、光伏和电价不得为负")
64
65     n = len(load)
66     # g, c, d, q(弃光), r(紧急购电), E[0:n], z
67     g = slice(0, n)
68     c = slice(n, 2 * n)
69     d = slice(2 * n, 3 * n)
70     q = slice(3 * n, 4 * n)
71     r = slice(4 * n, 5 * n)
72     e = slice(5 * n, 6 * n + 1)
73     z = slice(6 * n + 1, 7 * n + 1)

```

```

74     nvar = 7 * n + 1
75
76     obj = np.zeros(nvar)
77     obj[g] = tariff
78     obj[r] = emergency_multiplier * tariff
79
80     lower = np.zeros(nvar)
81     upper = np.full(nvar, np.inf)
82     interval_limit = max_power_kw * step_hours
83     upper[c] = interval_limit
84     upper[d] = interval_limit
85     upper[q] = pv
86     upper[r] = np.inf if allow_emergency else 0.0
87     lower[e], upper[e] = soc_min, soc_max
88     lower[5 * n] = upper[5 * n] = initial_soc
89     if terminal_soc is not None:
90         lower[6 * n] = upper[6 * n] = terminal_soc
91     upper[z] = 1.0
92
93     # 供需平衡和 SOC 状态转移。
94     aeq = lil_matrix((2 * n, nvar), dtype=float)
95     beq = np.zeros(2 * n)
96     for t in range(n):
97         aeq[t, g.start + t] = 1.0
98         aeq[t, c.start + t] = -1.0
99         aeq[t, d.start + t] = 1.0
100        aeq[t, q.start + t] = -1.0
101        aeq[t, r.start + t] = 1.0
102        beq[t] = load[t] - pv[t]
103
104        row = n + t
105        aeq[row, e.start + t] = -1.0
106        aeq[row, e.start + t + 1] = 1.0
107        aeq[row, c.start + t] = -eta_c
108        aeq[row, d.start + t] = 1.0 / eta_d
109

```

```

110 # c_t <= M z_t, d_t <= M(1-z_t)。LP 对照时 z 连续。
111 aub = lil_matrix((2 * n, nvar), dtype=float)
112 bub = np.zeros(2 * n)
113 for t in range(n):
114     aub[t, c.start + t] = 1.0
115     aub[t, z.start + t] = -interval_limit
116     aub[n + t, d.start + t] = 1.0
117     aub[n + t, z.start + t] = interval_limit
118     bub[n + t] = interval_limit
119
120 integrality = np.zeros(nvar, dtype=int)
121 if binary_mode:
122     integrality[z] = 1
123
124 constraints = [
125     LinearConstraint(aeq.tocsr(), beq, beq),
126     LinearConstraint(aub.tocsr(), -np.inf, bub),
127 ]
128 started = perf_counter()
129 raw = milp(
130     obj,
131     integrality=integrality,
132     bounds=Bounds(lower, upper),
133     constraints=constraints,
134     options={"time_limit": time_limit, "mip_rel_gap": 1e-8},
135 )
136 elapsed = perf_counter() - started
137 if raw.x is None:
138     raise RuntimeError(f"MILP 求解失败: {raw.message}")
139
140 x = raw.x
141 grid = x[g]
142 charge = x[c]
143 discharge = x[d]
144 curtailment = x[q]
145 emergency = x[r]

```

```

146     soc = x[e]
147     mode = x[z]
148     balance_residual = grid + pv + discharge + emergency - load -
charge - curtailment
149     soc_residual = soc[1:] - soc[:-1] - eta_c * charge + discharge /
eta_d
150     return DispatchResult(
151         grid=grid,
152         charge=charge,
153         discharge=discharge,
154         curtailment=curtailment,
155         emergency=emergency,
156         soc=soc,
157         mode=mode,
158         objective=float(raw.fun),
159         solve_seconds=elapsed,
160
max_balance_residual=float(np.max(np.abs(balance_residual))),
161         max_soc_residual=float(np.max(np.abs(soc_residual))),
162         success=bool(raw.success),
163         message=str(raw.message),
164     )

```

data_loader.py 问题一附件数据读取与单位转换

```

1  """2026 年 C 题附件数据读取与单位转换。"""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10
11  ROOT = Path(__file__).resolve().parents[1]
12  DATA_DIR = ROOT / "problems" / "选题 C_2026 正式" / "附件"

```

```

13 STEP_HOURS = 1.0 / 6.0
14
15
16 def load_q1() -> dict[str, np.ndarray]:
17     frame = pd.read_excel(DATA_DIR / "附件 1.xlsx")
18     required = ["时间", "电价", "小区负载", "光伏发电预测功率"]
19     if list(frame.columns) != required:
20         raise ValueError(f"附件 1 列名异常: {list(frame.columns)}")
21     if frame.shape[0] != 144 or frame.isna().any().any():
22         raise ValueError("附件 1 应包含 144 个完整的 10 分钟时段")
23     return {
24         "time": frame["时间"].astype(str).to_numpy(),
25         "price": frame["电价"].to_numpy(float),
26         "load_kw": frame["小区负载"].to_numpy(float),
27         "pv_kw": frame["光伏发电预测功率"].to_numpy(float),
28         "load_kwh": frame["小区负载"].to_numpy(float) * STEP_HOURS,
29         "pv_kwh": frame["光伏发电预测功率"].to_numpy(float) *
30         STEP_HOURS,
31     }

```

formal_data_loader.py 全年数据读取、字段校验与单位转换

```

1 """2026 C 题正式数据读取器。
2
3 功能：读取附件 1、附件 2 并执行结构、缺失和单位检查。
4 输入：problems/选题 C_2026 正式/附件/附件 1.xlsx、附件 2.xlsx
5 输出：内存中的日×时段数组；功率同时换算为 10 分钟电量。
6 运行：由 q2_model.py 导入。
7 """
8
9 from __future__ import annotations
10
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15

```

```

16
17 ROOT = Path(__file__).resolve().parents[1]
18 DATA_DIR = ROOT / "problems" / "选题 C_2026 正式" / "附件"
19 STEP_HOURS = 1.0 / 6.0
20 N_DAYS = 365
21 N_STEPS = 144
22
23 # .....
24 def _read_daily_sheet(path: Path, sheet_name: str) ->
    tuple[np.ndarray, np.ndarray]:
25     """读取附件 2 的一个 365×144 宽表, 返回日期与数值矩阵。"""
26
27     frame = pd.read_excel(path, sheet_name=sheet_name)
28     if frame.shape != (N_DAYS, N_STEPS + 1):
29         raise ValueError(
30             f"{path.name}/{sheet_name}形状应为(365,145), 实际为
31             {frame.shape}"
32         )
33     if frame.iloc[:, 1:].isna().any().any():
34         raise ValueError(f"{path.name}/{sheet_name}存在缺失数值")
35     dates = pd.to_datetime(frame.iloc[:, 0],
36                             errors="raise").to_numpy()
37     expected = pd.date_range("2025-01-01", "2025-12-31",
38                               freq="D").to_numpy()
39     if not np.array_equal(dates.astype("datetime64[D]"),
40                             expected.astype("datetime64[D]")):
41         raise ValueError(f"{path.name}/{sheet_name}日期不连续或不是
42                             2025 全年")
43     values = frame.iloc[:, 1:].to_numpy(dtype=float)
44     if not np.isfinite(values).all() or np.any(values < 0):
45         raise ValueError(f"{path.name}/{sheet_name}含非有限值或负功率
46                             ")
47     return dates, values
48
49 def load_typical_day() -> dict[str, np.ndarray]:

```

```

45     """读取附件 1 的固定电价与典型日曲线。"""
46
47     path = DATA_DIR / "附件 1.xlsx"
48     frame = pd.read_excel(path)
49     expected_columns = ["时间", "电价", "小区负载", "光伏发电预测功率"]
50     if list(frame.columns) != expected_columns or frame.shape[0] !=
N_STEPS:
51         raise ValueError("附件 1 列结构或时段数异常")
52     if frame.isna().any().any():
53         raise ValueError("附件 1 存在缺失值")
54     price = frame["电价"].to_numpy(dtype=float)
55     load_kw = frame["小区负载"].to_numpy(dtype=float)
56     pv_kw = frame["光伏发电预测功率"].to_numpy(dtype=float)
57     if np.any(price < 0) or np.any(load_kw < 0) or np.any(pv_kw <
0):
58         raise ValueError("附件 1 不应出现负的价格、负荷或光伏功率")
59     return {
60         "time": frame["时间"].astype(str).to_numpy(),
61         "price": price,
62         "load_kw": load_kw,
63         "pv_kw": pv_kw,
64         "load_kwh": load_kw * STEP_HOURS,
65         "pv_kwh": pv_kw * STEP_HOURS,
66         "net_kwh": (load_kw - pv_kw) * STEP_HOURS,
67     }
68
69
70 def load_q2_data() -> dict[str, np.ndarray]:
71     """读取 Q2 全年实际负荷/光伏，统一为 kW 与每时段 kWh。"""
72
73     path = DATA_DIR / "附件 2.xlsx"
74     dates_l, load_kw = _read_daily_sheet(path, "小区负载")
75     dates_s, pv_kw = _read_daily_sheet(path, "光伏发电实际功率")
76     if not np.array_equal(dates_l, dates_s):
77         raise ValueError("附件 2 两个工作表的日期不一致")

```

```

78     load_kwh = load_kw * STEP_HOURS
79     pv_kwh = pv_kw * STEP_HOURS
80     return {
81         "dates": dates_1.astype("datetime64[D]"),
82         "load_kw": load_kw,
83         "pv_kw": pv_kw,
84         "load_kwh": load_kwh,
85         "pv_kwh": pv_kwh,
86         "net_kwh": load_kwh - pv_kwh,
87     }

```

causal_forecasts.py 因果日前净负荷预测与整日残差情景生成

```

1  """Q2 因果日前净负荷预测与整日残差情景生成。
2
3  所有函数均通过 history_end 明确限制标签可见范围。任何目标日的真实值只在
4  该日完成后进入残差库。候选包括季节加权朴素法 N1 与岭回归 N3。
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10
11 import numpy as np
12 from sklearn.linear_model import Ridge
13 from sklearn.pipeline import make_pipeline
14 from sklearn.preprocessing import StandardScaler
15
16
17 ALPHA_GRID = (0.0, 0.25, 0.5, 0.75, 1.0)
18 RIDGE_LAMBDA_GRID = (0.01, 0.1, 1.0, 10.0, 100.0)
19 RIDGE_WINDOW_GRID = (56, 112, 10000)
20
21
22 # .....
23 class ForecastChoice:
24     method: str

```

```

25     params: dict[str, float | int | str]
26     validation_wape: float
27     validation_mae: float
28     validation_days: int
    # .....
36 def seasonal_forecast(
37     net: np.ndarray,
38     day: int,
39     alpha: float,
40     fallback: np.ndarray,
41     *,
42     recursive_lag1: np.ndarray | None = None,
43 ) -> np.ndarray:
44     """N1 预测; day 可以等于历史截止日以后的第一天。"""
45
46     if day <= 0:
47         return np.asarray(fallback, dtype=float).copy()
48     lag1 = recursive_lag1 if recursive_lag1 is not None else net[day
49 - 1]
50     lag7 = _safe_lag(net, day, 7, lag1)
51     return alpha * np.asarray(lag1) + (1.0 - alpha) *
52         np.asarray(lag7)
    # .....
101 def fit_ridge(
102     net: np.ndarray,
103     load: np.ndarray,
104     pv: np.ndarray,
105     history_end: int,
106     lam: float,
107     window: int,
108     fallback: np.ndarray,
109 ):
110     """在[0, history_end)日拟合 N3, 窗口和标准化均仅用过去。"""
111
112     train_start = max(1, history_end - int(window))
113     train_days = list(range(train_start, history_end))

```

```

114     if len(train_days) < 7:
115         return None
116     x = np.vstack([_day_features(net, load, pv, d, fallback) for d
117                    in train_days])
118     y = np.concatenate([net[d] for d in train_days])
119     model = make_pipeline(StandardScaler(), Ridge(alpha=float(lam),
120 fit_intercept=True))
121     model.fit(x, y)
122     return model
123
124 def ridge_forecast(
125     model,
126     net: np.ndarray,
127     load: np.ndarray,
128     pv: np.ndarray,
129     day: int,
130     fallback: np.ndarray,
131     *,
132     lag1_override: np.ndarray | None = None,
133 ) -> np.ndarray:
134     if model is None:
135         return seasonal_forecast(net, day, 1.0, fallback,
136 recursive_lag1=lag1_override)
137     x = _day_features(net, load, pv, day, fallback,
138 lag1_override=lag1_override)
139     return np.asarray(model.predict(x), dtype=float)
140
141 # .....
142
143 def select_month_model(
144     net: np.ndarray,
145     load: np.ndarray,
146     pv: np.ndarray,
147     month_start: int,
148     fallback: np.ndarray,
149 ) -> ForecastChoice:
150     """用月首前最近 14 日作严格时间验证，选择 N1 或 N3。

```

岭回归在验证段开始前一次拟合，因此其系数对 14 个验证原点均为过去信息；
滞后特征则按各验证日当时已完成的数据逐日构造。

```
"""
val_start = max(1, month_start - 14)
val_days = list(range(val_start, month_start))
if not val_days:
    return ForecastChoice("N1", {"alpha": 1.0}, float("nan"),
float("nan"), 0)

candidates: list[tuple[float, float, int, str, dict[str, float |
int | str]]] = []
for alpha in ALPHA_GRID:
    preds = np.vstack([seasonal_forecast(net, d, alpha,
fallback) for d in val_days])
    wape, mae = _metrics(net[val_days], preds)
    candidates.append((wape, mae, 1, "N1", {"alpha": alpha}))

train_end = val_start
for window in RIDGE_WINDOW_GRID:
    for lam in RIDGE_LAMBDA_GRID:
        model = fit_ridge(net, load, pv, train_end, lam, window,
fallback)
        if model is None:
            continue
        preds = np.vstack(
            [ridge_forecast(model, net, load, pv, d, fallback)
for d in val_days]
        )
        wape, mae = _metrics(net[val_days], preds)
        candidates.append(
            (wape, mae, 3, "N3", {"lambda": lam, "window_days":
int(window)})
        )
candidates.sort(key=lambda row: (row[0], row[2], row[1]))
```

```

185     best = candidates[0]
186     # 差异不足 1%时按预先规则选更简单的 N1。
187     near = [row for row in candidates if row[0] <= best[0] * 1.01]
188     chosen = min(near, key=lambda row: (row[2], row[0], row[1]))
189     return ForecastChoice(chosen[3], chosen[4], chosen[0],
        chosen[1], len(val_days))
# .....
252 def residual_scenarios(
253     point: np.ndarray,
254     residuals: np.ndarray,
255     *,
256     scenario_count: int | None = None,
257 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
258     """用整日残差向量作等概率分位抽样，保留 144 维日内相关性。"""
259
260     point = np.asarray(point, dtype=float)
261     if residuals.size == 0:
262         return point[None, :], np.ones(1), np.array([], dtype=int)
263     residuals = np.asarray(residuals, dtype=float)
264     count = scenario_count or choose_scenario_count(residuals)
265     count = min(max(1, int(count)), len(residuals))
266     order = np.argsort(residuals.sum(axis=1))
267     idx_sorted = np.linspace(0, len(residuals) - 1,
        count).round().astype(int)
268     selected = order[idx_sorted]
269     scenarios = point[None, :] + residuals[selected]
270     probs = np.full(count, 1.0 / count)
271     return scenarios, probs, selected

```

q34_helpers.py 问题三、四共用的滚动更新与调整结算

```

1  """Q3/Q4 共用的因果预测、滚动调整 MILP 与回测工具。
2
3  所有功率输入在读入后乘 1/6，内部决策量均为每十分钟电量 (kWh) 。
4  附件中的未来真实负荷、光伏和价格只进入事后结算，不进入预测函数。
5  """
6

```

```

7 from __future__ import annotations
8
9 from dataclasses import dataclass
10 from pathlib import Path
11 from time import perf_counter
12 from typing import Callable, Iterable
13
14 import numpy as np
15 import pandas as pd
16 from scipy.optimize import Bounds, LinearConstraint, milp
17 from scipy.sparse import lil_matrix
18
19 from common_milp import solve_deterministic_dispatch
20
21
22 STEP_HOURS = 1.0 / 6.0
23 SLOTS_PER_DAY = 144
24 ORIGIN_SLOTS = (0, 36, 72, 108)
25 SOC_MIN = 1200.0
26 SOC_MAX = 10800.0
27 INTERVAL_LIMIT = 5000.0 * STEP_HOURS
28 ETA_C = 0.9
29 ETA_D = 0.9
30
31 # .....
85 def pv_forecast_10min(data: CData, day: int, origin: int) ->
    np.ndarray:
86     """仅用本次发布的 24 个整点预测和发布时刻已观测锚点插值。"""
87     oi = ORIGIN_SLOTS.index(origin)
88     hourly = data.pv_hourly_forecast[day, oi]
89     if origin == 0:
90         anchor = data.pv_actual[day - 1, -1] if day > 0 else
data.typical_pv[-1]
91     else:
92         anchor = data.pv_actual[day, origin - 1]
93     horizon = SLOTS_PER_DAY - origin

```

```

94     x = np.arange(25, dtype=float)
95     y = np.r_[anchor, hourly]
96     target = np.arange(1, horizon + 1, dtype=float) / 6.0
97     return np.maximum(np.interp(target, x, y), 0.0)
98
99
100 def _seasonal_curve(values: np.ndarray, typical: np.ndarray, day:
    int, alpha: float) -> np.ndarray:
101     if day <= 0:
102         return typical.copy()
103     lag1 = values[day - 1]
104     lag7 = values[day - 7] if day >= 7 else typical
105     return alpha * lag1 + (1.0 - alpha) * lag7
106
107
108 def predict_load(
109     data: CData,
110     day: int,
111     origin: int,
112     params: tuple[float, int, float],
113 ) -> np.ndarray:
114     """日前季节基线加仅由已实现前缀计算的稳健偏差校正。"""
115     alpha, window, rho = params
116     baseline = _seasonal_curve(data.load, data.typical_load, day,
    alpha)
117     future = baseline[origin:].copy()
118     if origin > 0:
119         lo = max(0, origin - window)
120         correction = float(np.median(data.load[day, lo:origin] -
    baseline[lo:origin]))
121         lead = np.arange(1, len(future) + 1, dtype=float)
122         future += np.power(rho, lead / 36.0) * correction
123     return np.maximum(future, 0.0)
124
125 # .....
232 def solve_adjustment_dispatch(
233     load_kwh: np.ndarray,

```

```

234     pv_kwh: np.ndarray,
235     price: np.ndarray,
236     initial_soc: float,
237     old_grid: np.ndarray,
238     *,
239     terminal_soc: float | None = None,
240     binary_mode: bool = True,
241     time_limit: float = 120.0,
242 ) -> dict[str, np.ndarray | float | bool | str]:
243     """最小化未来增减购结算与预测紧急购电费。"""
244     load = np.asarray(load_kwh, float)
245     pv = np.asarray(pv_kwh, float)
246     tariff = np.asarray(price, float)
247     old = np.asarray(old_grid, float)
248     n = len(load)
249     if not (len(pv) == len(tariff) == len(old) == n):
250         raise ValueError("调整模型各输入长度不一致")
251
252     # g,c,d,w,r,E,z,inc,dec
253     g = slice(0, n)
254     c = slice(n, 2 * n)
255     d = slice(2 * n, 3 * n)
256     w = slice(3 * n, 4 * n)
257     emergency = slice(4 * n, 5 * n)
258     e = slice(5 * n, 6 * n + 1)
259     z = slice(6 * n + 1, 7 * n + 1)
260     inc = slice(7 * n + 1, 8 * n + 1)
261     dec = slice(8 * n + 1, 9 * n + 1)
262     nvar = 9 * n + 1
263
264     obj = np.zeros(nvar)
265     obj[emergency] = 5.0 * tariff
266     obj[inc] = 1.5 * tariff
267     obj[dec] = -0.5 * tariff
268     lower = np.zeros(nvar)
269     upper = np.full(nvar, np.inf)

```

```

270 upper[c] = INTERVAL_LIMIT
271 upper[d] = INTERVAL_LIMIT
272 upper[w] = pv
273 lower[e], upper[e] = SOC_MIN, SOC_MAX
274 lower[e.start] = upper[e.start] = initial_soc
275 if terminal_soc is not None:
276     lower[e.stop - 1] = upper[e.stop - 1] = terminal_soc
277 upper[z] = 1.0
278
279 # 能量平衡、SOC、计划差额恒等式。
280 aeq = lil_matrix((3 * n, nvar), dtype=float)
281 beq = np.zeros(3 * n)
282 for t in range(n):
283     aeq[t, g.start + t] = 1.0
284     aeq[t, c.start + t] = -1.0
285     aeq[t, d.start + t] = 1.0
286     aeq[t, w.start + t] = -1.0
287     aeq[t, emergency.start + t] = 1.0
288     beq[t] = load[t] - pv[t]
289
290     row = n + t
291     aeq[row, e.start + t] = -1.0
292     aeq[row, e.start + t + 1] = 1.0
293     aeq[row, c.start + t] = -ETA_C
294     aeq[row, d.start + t] = 1.0 / ETA_D
295
296     row = 2 * n + t
297     aeq[row, g.start + t] = 1.0
298     aeq[row, inc.start + t] = -1.0
299     aeq[row, dec.start + t] = 1.0
300     beq[row] = old[t]
301
302 aub = lil_matrix((2 * n, nvar), dtype=float)
303 bub = np.zeros(2 * n)
304 for t in range(n):
305     aub[t, c.start + t] = 1.0

```

```

306     aub[t, z.start + t] = -INTERVAL_LIMIT
307     aub[n + t, d.start + t] = 1.0
308     aub[n + t, z.start + t] = INTERVAL_LIMIT
309     bub[n + t] = INTERVAL_LIMIT
310
311     integrality = np.zeros(nvar, dtype=int)
312     if binary_mode:
313         integrality[z] = 1
314     started = perf_counter()
315     raw = milp(
316         obj,
317         integrality=integrality,
318         bounds=Bounds(lower, upper),
319         constraints=[
320             LinearConstraint(aeq.tocsr(), beq, beq),
321             LinearConstraint(aub.tocsr(), -np.inf, bub),
322         ],
323         options={"time_limit": time_limit, "mip_rel_gap": 1e-8},
324     )
325     elapsed = perf_counter() - started
326     if raw.x is None:
327         raise RuntimeError(f"调整 MILP 求解失败: {raw.message}")
328     x = raw.x
329     balance = x[g] + pv + x[d] + x[emergency] - load - x[c] - x[w]
330     soc_res = x[e][1:] - x[e][: -1] - ETA_C * x[c] + x[d] / ETA_D
331     return {
332         "grid": x[g],
333         "charge": x[c],
334         "discharge": x[d],
335         "curtailment": x[w],
336         "predicted_emergency": x[emergency],
337         "soc": x[e],
338         "increase": x[inc],
339         "decrease": x[dec],
340         "objective": float(raw.fun),
341         "solve_seconds": elapsed,

```

```
342     "success": bool(raw.success),
343     "message": str(raw.message),
344     "max_balance_residual": float(np.max(np.abs(balance))),
345     "max_soc_residual": float(np.max(np.abs(soc_res))),
346 }
```

verify_results.py 四问约束与费用的独立复核

```
1  """对四问结果执行独立数值复核并生成逐问门禁所需追溯文件。"""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from pathlib import Path
8
9  import numpy as np
10
11
12  ROOT = Path(__file__).resolve().parents[1]
13  STATE = ROOT / "state" / "agent_outputs"
14  RESULTS = ROOT / "results"
15  DATA = ROOT / "problems" / "选题 C_2026 正式" / "附件"
16  # .....
17
18  def sha(path: Path) -> str:
19      h = hashlib.sha256(path.read_bytes()).hexdigest()
20      return h.upper()
21
22
23  def write_json(path: Path, value: dict) -> None:
24      path.write_text(json.dumps(value, ensure_ascii=False, indent=2),
25                      encoding="utf-8")
26
27
28  def registry(qid, claims):
29      write_json(STATE / f"{qid.lower()}_claim_registry.json",
30               {"sub_question": qid, "claims": claims})
```

```

29
30
31 def claim(cid, rid, statement, result_path, symbol, unit, test,
    hashes):
32     return {
33         "claim_id": cid, "requirement_id": rid, "statement":
statement,
34         "result_json_path": result_path if
result_path.startswith("$.") else "$." + result_path,
35         "code_symbol": symbol, "unit": unit,
36         "validation_test": test, "raw_input_hashes": hashes,
"status": "verified",
37     }
38
39
40 def normalize_registries():
41     """把登记表中的裸路径补齐为证据门禁要求的 JSONPath (含无生成脚本的 Q1
表) 。”"""
42     for path in sorted(STATE.glob("q*_claim_registry.json")):
43         data = json.loads(path.read_text(encoding="utf-8"))
44         changed = False
45         for item in data.get("claims", []):
46             current = item.get("result_json_path", "")
47             if current and not current.startswith("$."):
48                 item["result_json_path"] = "$." + current
49                 changed = True
50         if changed:
51             write_json(path, data)
52     # .....
81 def verify_q3():
82     z = np.load(RESULTS / "q3_detail.npz", allow_pickle=True)
83     dates = np.asarray(z["dates"]).astype(str)
84     g, c, d, r, w, soc = z["effective_grid"], z["charge"],
z["discharge"], z["emergency"], z["curtailment"], z["soc"]
85     from q34_helpers import load_c_data
86     data = load_c_data(DATA)

```

```

87     balance = g + data.pv_actual[:len(dates)] + d + r -
data.load[:len(dates)] - c - w
88     checks = {
89         "365_days_present": len(dates) == 365,
90         "official_window_334_days": int(np.sum(dates >= "2025-02-
01")) == 334,
91         "actual_balance_residual_le_1e-6":
float(np.max(np.abs(balance))) <= 1e-6,
92         "soc_transition_residual_le_1e-6":
float(np.max(np.abs(soc[:,1:] - soc[:, :-1] - 0.9*c + d/0.9))) <=
1e-6,
93         "soc_linkage_le_1e-6": float(np.max(np.abs(soc[1:,0] -
soc[:, -1, -1]))) <= 1e-6,
94         "soc_bounds": float(soc.min()) >= 1200 - 1e-6 and
float(soc.max()) <= 10800 + 1e-6,
95         "charge_discharge_mutex": int(np.sum((c > 1e-7) & (d > 1e-
7))) == 0,
96         "cost_ledger": np.allclose(z["planned_cost"] +
z["adjustment_cost"] + z["emergency_cost"], z["total_cost"],
atol=1e-5),
97     }
98     write_json(STATE / "q3_verification.json", {"status": "passed"
if all(checks.values()) else "failed", "requirement_coverage":
"100%", "interface_status": "passed", "checks": checks})
99     h = {"附件 1.xlsx": sha(DATA / "附件 1.xlsx"), "附件 2.xlsx":
sha(DATA / "附件 2.xlsx"), "附件 3.xlsx": sha(DATA / "附件
3.xlsx")}
100     registry("Q3", [
101         claim("Q3-TOTAL", "R3", "四次滚动更新策略的正式期总成本",
"key_results.total_cost_yuan", "simulate_mpc", "元", "计划、逐次
调整与紧急费用恒等式", h),
102         claim("Q3-UPDATE", "R3", "是否保留各发布时刻由全年消融结果决定
", "update_ablation.0_6_12_18.total_cost_yuan", "ORIGIN_SLOTS",
"元", "0、0/6、0/6/12、0/6/12/18 同日配对比较", h),

```

```

103         claim("Q3-EMERGENCY", "R3", "滚动策略正式期紧急购电量",
104               "key_results.emergency_energy_kwh", "emergency", "kWh", "实际供需
            缺口逐时段复算", h),
            ])

```

export_result_workbooks.py 计算结果工作簿导出

```

1  """把经验证的数组回填为附件 5 要求的五份结果工作簿。"""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import numpy as np
8  from openpyxl import Workbook, load_workbook
9  from openpyxl.styles import Alignment, Border, Font, PatternFill,
        Side
10 from openpyxl.utils import get_column_letter
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 TEMPLATE_DIR = ROOT / "problems" / "选题 C_2026 正式" / "附件" / "附件
        5"
15 OUT_DIR = ROOT / "results" / "submission_results"
16 OUT_DIR.mkdir(parents=True, exist_ok=True)
17 TOL = 1e-7
18 # .....
19
20 def aggregate_blocks(charge, discharge, soc):
21     rows = []
22     for start in range(0, 144, 24):
23         block_index = start // 24
24         state_time = "0:00" if block_index == 0 else ("24:00" if
25 block_index == 1 else None)
26         state_value = float(soc[0]) if block_index == 0 else
27 (float(soc[-1]) if block_index == 1 else None)
28         rows.append((
29             f"{start // 6}:00-{(start + 24) // 6}:00",

```

```

84         float(charge[start:start + 24].sum()),
85         float(discharge[start:start + 24].sum()),
86         state_time,
87         state_value,
88     ))
89     return rows
90
91
92 def emergency_events(values, labels):
93     events = []
94     i = 0
95     while i < 144:
96         if values[i] <= TOL:
97             i += 1
98             continue
99         j = i
100        while j + 1 < 144 and values[j + 1] > TOL:
101            j += 1
102            start = labels[i].split("-")[0]
103            end = labels[j].split("-")[-1]
104            events.append((f"{start}-{end}", float(values[i:j +
105            1].sum()))))
105            i = j + 1
106    return events

```

export_matlab_visual_data.py MATLAB 绘图数据导出

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  import numpy as np
7  import pandas as pd
8
9
10 ROOT = Path(__file__).resolve().parents[1]

```

```

11 OUT = ROOT / "results" / "matlab_plot_data"
12 OUT.mkdir(parents=True, exist_ok=True)
13
14 # .....
15 def export_q2() -> None:
16     z = np.load(ROOT / "results" / "q2_detail.npz",
17                 allow_pickle=True)
18     dates = pd.to_datetime(z["dates"])
19     start = int(z["official_start_index"][0])
20     soc_path = z["soc_start_kwh"][:, None] + np.cumsum(
21         0.9 * z["planned_charge_kwh"] - z["planned_discharge_kwh"] /
22         0.9,
23         axis=1,
24     )
25     daily = pd.DataFrame(
26         {
27             "date": dates[start:],
28             "planned_cost_yuan": z["planned_cost_yuan"][start:],
29             "emergency_cost_yuan": z["emergency_cost_yuan"][start:],
30             "emergency_energy_kwh":
31             z["actual_emergency_kwh"][start:].sum(axis=1),
32             "soc_start_kwh": z["soc_start_kwh"][start:],
33             "soc_end_kwh": z["soc_end_kwh"][start:],
34             "soc_min_kwh": np.minimum(z["soc_start_kwh"],
35             soc_path.min(axis=1))[start:],
36             "soc_max_kwh": np.maximum(z["soc_start_kwh"],
37             soc_path.max(axis=1))[start:],
38             "soc_link_residual_kwh": np.r_[0.0,
39             z["soc_start_kwh"][1:] - z["soc_end_kwh"][:-1]][start:],
40         }
41     )
42     daily.to_csv(OUT / "q2_daily_summary.csv", index=False,
43                 encoding="utf-8-sig")
44     monthly =
45     daily.set_index("date").resample("ME").sum(numeric_only=True).re
46     set_index()

```

```

38     monthly.to_csv(OUT / "q2_monthly_summary.csv", index=False,
encoding="utf-8-sig")
39
40     chosen = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-
21"]
41     parts = []
42     for day in chosen:
43         i = int(np.flatnonzero(dates.strptime("%Y-%m-%d") ==
day)[0])
44         parts.append(
45             pd.DataFrame(
46                 {
47                     "date": day,
48                     "interval": np.arange(144),
49                     "hour": np.arange(144) / 6.0,
50                     "actual_net_kw": z["actual_net_kwh"][i] * 6,
51                     "forecast_net_kw": z["point_forecast_net_kwh"][i]
* 6,
52                     "planned_grid_kw": z["planned_grid_kwh"][i] * 6,
53                     "emergency_kw": z["actual_emergency_kwh"][i] * 6,
54                 }
55             )
56         )
57     pd.concat(parts, ignore_index=True).to_csv(
58         OUT / "q2_representative_days.csv", index=False,
encoding="utf-8-sig"
59     )

```

run_all.py 统一核验与完整复算入口

```

1  """C 题统一运行入口：求解、核验与结果导出。
2
3  用法：
4      python run_all.py          # 冒烟求解并核验
5      python run_all.py --full   # 全年完整复算并核验
6      python run_all.py --check  # 仅核验已有结果
7      python run_all.py --stage solve # 仅运行四问求解程序

```

```

8     python run_all.py --stage export # 仅导出结果工作簿与 MATLAB 数据
9     """
10
11 from __future__ import annotations
12
13 import argparse
14 import subprocess
15 import sys
16 import time
17 from pathlib import Path
18
19
20 ROOT = Path(__file__).resolve().parents[1]
21 CODE = ROOT / "code"
22
23 SOLVE_STEPS = [
24     ("q1_model.py", "问题一确定性调度"),
25     ("q2_model.py", "问题二日前两阶段调度"),
26     ("q3_model.py", "问题三滚动调度"),
27     ("q4_model.py", "问题四实时价格策略"),
28 ]
29 VERIFY_STEPS = [("verify_results.py", "四问独立数值复核")]
30 EXPORT_STEPS = [
31     ("export_result_workbooks.py", "导出五份结果工作簿"),
32     ("export_matlab_visual_data.py", "导出 MATLAB 绘图数据"),
33 ]
34 # .....
35
36 def run_step(script: str, note: str, extra_args: list[str] | None =
    None) -> None:
37     print(f"\n{'=' * 68}\n► {script} — {note}\n{'=' * 68}",
    flush=True)
38     started = time.time()
39     result = subprocess.run(
40         [sys.executable, script, *(extra_args or [])], cwd=CODE,
    check=False
41     )

```

```
42     if result.returncode != 0:
43         raise SystemExit(f"✗ {script} 失败 (退出码
{result.returncode}) ")
44     print(f"✓ 完成, 用时 {time.time() - started:.1f}s", flush=True)
45
46
47 def main() -> None:
48     parser = argparse.ArgumentParser(description="C 题模型复算与核验
")
49     parser.add_argument("--full", action="store_true", help="执行全
年完整回测")
50     parser.add_argument("--check", action="store_true", help="仅核验
已有结果")
51     parser.add_argument(
52         "--stage", choices=["all", "solve", "verify", "export"],
default="all"
53     )
54     args = parser.parse_args()
55
56     (ROOT / "results").mkdir(parents=True, exist_ok=True)
57     (ROOT / "state" / "agent_outputs").mkdir(parents=True,
exist_ok=True)
58
59     if args.check:
60         for item in VERIFY_STEPS:
61             run_step(*item)
62             print("\n☑ 既有结果核验完成")
63         return
64
65     if args.stage in {"all", "solve"}:
66         for script, note in SOLVE_STEPS:
67             extra = ["--full"] if args.full and script in
{"q3_model.py", "q4_model.py"} else []
68             run_step(script, note, extra)
69
70     if args.stage in {"all", "verify"}:
```

```

71         for item in VERIFY_STEPS:
72             run_step(*item)
73
74     if args.stage in {"all", "export"}:
75         for item in EXPORT_STEPS:
76             run_step(*item)
77
78     print("\n☑ 指定流程全部完成")

```

plot_q1_dispatch_matlab_preview.m 问题一调度、储能动作与 SOC 轨迹

```

1  %% 问题一：典型日功率调度与储能状态（国奖论文表达方式修订图）
2  % 输入：results/q1_detail.csv
3  % 输出：
4      results/figures_matlab/q1_dispatch_matlab_awardstyle.{png,pdf,fig}
5
6  % 运行：matlab -batch "run('D:/数模 workflows /code/plot_q1_dispatch_matlab_preview.m')"
7  # .....
8
9  t = (T.slot - 0.5) / 6;
10 kwFactor = 6; % 10 分钟电量(kWh)换算为该时段平均功率(kW)
11 loadKW = T.load_kwh * kwFactor;
12 pvKW = T.pv_kwh * kwFactor;
13 gridKW = T.grid_kwh * kwFactor;
14 chargeKW = T.charge_kwh * kwFactor;
15 dischargeKW = T.discharge_kwh * kwFactor;
16 socPct = 100 * T.soc_end_kwh / 12000;
17 price = T.price_yuan_per_kwh;
18
19 % 颜色采用低饱和、色盲相对友好的竞赛论文配色。
20 cNavy = [30, 78, 121] / 255;
21 cTeal = [42, 157, 143] / 255;
22 cGold = [233, 167, 55] / 255;
23 cCoral = [220, 92, 72] / 255;
24 cGray = [92, 102, 112] / 255;
25 cGrid = [220, 225, 230] / 255;
26 # .....

```

```

37 fig = figure('Color', 'w', 'Units', 'centimeters', ...
38     'Position', [2, 2, 18.2, 14.8], 'Renderer', 'painters');
39 layout = tiledlayout(fig, 3, 1, 'TileSpacing', 'compact',
    'Padding', 'compact');
40
41 % (a) 负荷、光伏与外网购电的时序关系。
42 ax1 = nexttile(layout, 1);
43 hold(ax1, 'on');
44 plot(ax1, t, loadKW, '-', 'Color', cNavy, 'LineWidth', 1.75);
45 plot(ax1, t, pvKW, '-', 'Color', cGold, 'LineWidth', 1.75);
46 stairs(ax1, t, gridKW, '--', 'Color', cTeal, 'LineWidth', 1.45);
47 ylabel(ax1, '功率 / kW');
48 legend(ax1, {'居民负荷', '光伏出力', '外网购电'}, ...
49     'Location', 'northoutside', 'Orientation', 'horizontal', ...
50     'Box', 'off', 'NumColumns', 3);
51 text(ax1, 0.01, 0.91, '(a)', 'Units', 'normalized', ...
52     'FontName', 'Times New Roman', 'FontWeight', 'bold', 'FontSize',
    10.5);
53
54 % (b) 储能动作：放电为正、充电为负；柱宽对应 10 分钟执行区间。
55 ax2 = nexttile(layout, 2);
56 hold(ax2, 'on');
57 hBar = bar(ax2, t, [dischargeKW, -chargeKW], 1.0, 'stacked', ...
58     'EdgeColor', 'none');
59 hBar(1).FaceColor = cCoral;
60 hBar(1).FaceAlpha = 0.88;
61 hBar(2).FaceColor = cTeal;
62 hBar(2).FaceAlpha = 0.82;
63 yline(ax2, 0, '-', 'Color', cGray, 'LineWidth', 0.85);
64 ylabel(ax2, '储能功率 / kW');
65 legend(ax2, {'放电', '充电'}, 'Location', 'northoutside', ...
66     'Orientation', 'horizontal', 'Box', 'off', 'NumColumns', 2);
67 text(ax2, 0.01, 0.91, '(b)', 'Units', 'normalized', ...
68     'FontName', 'Times New Roman', 'FontWeight', 'bold', 'FontSize',
    10.5);
69

```

```

70 % (c) SOC 与分时电价共同解释充放电动作的经济原因。
71 ax3 = nexttile(layout, 3);
72 yyaxis(ax3, 'left');
73 plot(ax3, t, socPct, '-', 'Color', cNavy, 'LineWidth', 1.9);
74 yline(ax3, 10, ':', 'Color', cGray, 'LineWidth', 0.9);
75 yline(ax3, 90, ':', 'Color', cGray, 'LineWidth', 0.9);
76 ylabel(ax3, '储能荷电状态 SOC / %');
77 ylim(ax3, [5, 95]);
78
79 yyaxis(ax3, 'right');
80 stairs(ax3, t, price, '-', 'Color', cGold, 'LineWidth', 1.45);
81 ylabel(ax3, '电价 / (元·kWh^{-1})');
82 ax3.YAxis(1).Color = cGray;
83 ax3.YAxis(2).Color = cGray;
84 text(ax3, 0.01, 0.91, '(c)', 'Units', 'normalized', ...
85     'FontName', 'Times New Roman', 'FontWeight', 'bold', 'FontSize',
86     10.5);
87 # .....
88 axesList = [ax1, ax2, ax3];
89 for ax = axesList
90     xlim(ax, [0, 24]);
91     xticks(ax, 0:4:24);
92     ax.FontName = 'Microsoft YaHei';
93     ax.FontSize = 9.5;
94     ax.LineWidth = 0.8;
95     ax.TickDir = 'out';
96     ax.Box = 'off';
97     ax.XGrid = 'off';
98     ax.YGrid = 'on';
99     ax.GridColor = cGrid;
100     ax.GridAlpha = 0.42;
101     ax.MinorGridAlpha = 0.25;
102 end
103 ax1.XTickLabel = [];
104 ax2.XTickLabel = [];
105 xlabel(ax3, '时刻 / h');

```

```

106 linkaxes(axesList, 'x');
107
108 outBase = fullfile(outputDir, 'q1_dispatch_matlab_awardstyle');
109 exportgraphics(fig, [outBase '.png'], 'Resolution', 600,
    'BackgroundColor', 'white');
110 exportgraphics(fig, [outBase '.pdf'], 'ContentType', 'vector',
    'BackgroundColor', 'white');
111 savefig(fig, [outBase '.fig']);

```

plot_c_paper_figures_matlab.m 问题二至问题四证据图

```

1 %% C 题论文问题二至问题四证据图 (MATLAB 统一重绘)
2 % 数据由 export_matlab_visual_data.py 从已验证结果导出; 本脚本不重新求
    解模型。
3
4 # .....
11 fontCN = 'Microsoft YaHei';
12 cNavy = [30, 78, 121] / 255;
13 cBlue = [76, 120, 168] / 255;
14 cTeal = [42, 157, 143] / 255;
15 cGold = [233, 167, 55] / 255;
16 cCoral = [220, 92, 72] / 255;
17 cPurple = [111, 78, 143] / 255;
18 cGray = [92, 102, 112] / 255;
19 cLight = [221, 227, 232] / 255;
20 # .....
83 %% 问题三: 更新频率消融
84 A = readmatrix(fullfile(dataDir, 'q3_update_ablation.csv'),
    'Delimiter', ',');
85 labels = {'仅 0 时', '0/6 时', '0/6/12 时', '0/6/12/18 时'};
86 x = 1:size(A,1);
87 fig = figure('Color','w','Units','centimeters','Position',[2 2 18.2
    8.5],'Renderer','painters');
88 t1 =
    tiledlayout(fig,1,2,'TileSpacing','compact','Padding','compact')
    ;
89 ax1 = nexttile(t1,1); hold(ax1,'on');

```

```

90 v1=A(:,1)/1e4;
91 bar(ax1,x,v1,0.64,'FaceColor',cBlue,'EdgeColor','none');
92 for i=1: numel(x),
    text(ax1,x(i),v1(i)+2.5,sprintf('%.1f',v1(i)), 'HorizontalAlignme
        nt','center','FontSize',8.2); end
93 ylabel(ax1,'全年总费用 / 万元'); xticks(ax1,x);
    xticklabels(ax1,labels); xtickangle(ax1,16);
94 ylim(ax1,[min(v1)-18 max(v1)+18]); styleAxes(ax1,fontCN,cLight);
    text(ax1,0.02,0.94,'(a)','Units','normalized','FontWeight','bold
        ');
95 ax2 = nexttile(tl,2); hold(ax2,'on');
96 v2=A(:,5)/1e3;
97 bar(ax2,x,v2,0.64,'FaceColor',cCoral,'EdgeColor','none');
98 for i=1: numel(x),
    text(ax2,x(i),v2(i)+8,sprintf('%.1f',v2(i)), 'HorizontalAlignment
        ','center','FontSize',8.2); end
99 ylabel(ax2,'紧急购电量 / MWh'); xticks(ax2,x);
    xticklabels(ax2,labels); xtickangle(ax2,16);
100 ylim(ax2,[0 max(v2)*1.14]); styleAxes(ax2,fontCN,cLight);
    text(ax2,0.02,0.94,'(b)','Units','normalized','FontWeight','bold
        ');
101 saveTriple(fig,outDir,'q3_update_ablation_matlab'); close(fig);
    # .....
103 %% 问题三：滚动预测修正轨迹
104 U = readtable(fullfile(dataDir, 'q3_representative_updates.csv'),
    'VariableNamingRule','preserve');
105 days = unique(string(U.date),'stable');
106 fig = figure('Color','w','Units','centimeters','Position',[2 2 18.2
    13.0],'Renderer','painters');
107 tl =
    tiledlayout(fig,2,2,'TileSpacing','compact','Padding','compact')
    ;
108 forecastNames={'forecast_00_kw','forecast_06_kw','forecast_12_kw','
    forecast_18_kw'};
109 forecastColors={cBlue,cTeal,cGold,cPurple};
110 for i=1: numel(days)

```

```

111     ax=nexttile(tl,i); hold(ax,'on');
    S=U(string(U.date)==days(i),:);
112
    b=bar(ax,S.hour,S.emergency_kw,1.0,'FaceColor',cCoral,'EdgeColor',
    ',none'); b.FaceAlpha=0.25;
113     plot(ax,S.hour,S.actual_net_kw,'-
    ', 'Color',cNavy,'LineWidth',1.6);
114     for j=1:4
115         plot(ax,S.hour,S.(forecastNames{j}),'--
    ', 'Color',forecastColors{j},'LineWidth',1.0);
116     end
117     xline(ax,6,':','Color',cGray); xline(ax,12,':','Color',cGray);
    xline(ax,18,':','Color',cGray);
118
    text(ax,0.02,0.91,char(days(i)),'Units','normalized','FontName',
    fontCN,'FontWeight','bold','FontSize',9.2);
119     styleAxes(ax,fontCN,cLight); xlim(ax,[0 24]); xticks(ax,0:6:24);
120     if i>2, xlabel(ax,'时刻 / h'); else, ax.XTickLabel=[]; end
121     if mod(i,2)==1, ylabel(ax,'功率 / kW'); end
122 end
123 lg=legend(tl.Children(end),{'紧急购电','实际净负荷','0 时预测','6 时预
    测','12 时预测','18 时预测'}, ...
124     'Orientation','horizontal','Box','off','NumColumns',6);
    lg.Layout.Tile='north';
125 saveTriple(fig,outDir,'q3_forecast_updates_matlab'); close(fig);
    # .....
127 %% 问题四：总成本层级与价格信息价值
128 V = readmatrix(fullfile(dataDir, 'q4_information_value.csv'),
    'Delimiter', ',');
129 questionLabels = {'Q4-2','Q4-3'};
130 x=1:size(V,1);
131 fig = figure('Color','w','Units','centimeters','Position',[2 2 18.2
    8.6],'Renderer','painters');
132 tl=tiledlayout(fig,1,2,'TileSpacing','compact','Padding','compact')
    ;
133 ax1=nexttile(tl,1); hold(ax1,'on');

```

```

134 vals=V(:,2:4)/1e4;
135 b=bar(ax1,x,vals,'grouped','EdgeColor','none');
    b(1).FaceColor=cBlue; b(2).FaceColor=cGold;
    b(3).FaceColor=cGray;
136 ylabel(ax1,'全年总费用 / 万元'); xticks(ax1,x);
    xticklabels(ax1,questionLabels);
137 legend(ax1,{'因果在线','完全价格信息','全信息下界
    ','Box','off','Location','northoutside','NumColumns',3});
138 styleAxes(ax1,fontCN,cLight);
    text(ax1,0.02,0.94,'(a)','Units','normalized','FontWeight','bold
    ');
139 ax2=nexttile(tl,2); hold(ax2,'on');
140 gap=V(:,5)/1e4;
141 bar(ax2,x,gap,0.55,'FaceColor',cCoral,'EdgeColor','none');
142 for i=1:numel(x)
143     text(ax2,x(i),gap(i)+0.35,sprintf('%.2f 万元
    \n(%.3f%%)',gap(i),V(i,6)), ...
144
    'HorizontalAlignment','center','FontName',fontCN,'FontSize',8.8)
    ;
145 end
146 ylabel(ax2,'价格信息损失 / 万元'); xticks(ax2,x);
    xticklabels(ax2,questionLabels);
147 ylim(ax2,[0 max(gap)*1.30]); styleAxes(ax2,fontCN,cLight);
    text(ax2,0.02,0.94,'(b)','Units','normalized','FontWeight','bold
    ');
148 saveTriple(fig,outDir,'q4_information_value_matlab'); close(fig);
# .....
152 function styleAxes(ax,fontCN,cGrid)
153     ax.FontName=fontCN; ax.FontSize=9.2; ax.LineWidth=0.8;
    ax.TickDir='out'; ax.Box='off';
154     ax.XGrid='off'; ax.YGrid='on'; ax.GridColor=cGrid;
    ax.GridAlpha=0.46;
155 end
156
157 function saveTriple(fig,outDir,stem)

```

```
158     base=fullfile(outDir,stem);
159     exportgraphics(fig,[base
    '.png'],'Resolution',600,'BackgroundColor','white');
160     exportgraphics(fig,[base
    '.pdf'],'ContentType','vector','BackgroundColor','white');
161     savefig(fig,[base '.fig']);
```